

Tipos de datos

Java es un lenguaje de tipado fuerte, lo que significa que tienes que decirle al compilador exactamente qué tipo de información vas a guardar en cada "caja" antes de usarla.

1. `int`: Guarda números enteros sin decimales.

```
// Archivo: PruebaEnteros.java
public class PruebaEnteros {
    public static void main(String[] args) {
        // Declaramos obligatoriamente que es un 'int'
        int usuariosActivos = 45;
        usuariosActivos++; // El incremento de Java que tanto me gusta

        // Así imprimimos en la consola de Java:
        System.out.println("Usuarios en la terminal: " +
usuariosActivos);
    }
}
```

2. `double`: Almacena números con punto decimal de alta precisión.

```
// Archivo: PruebaDecimales.java
public class PruebaDecimales {
    public static void main(String[] args) {
        double precioProducto = 99.99;
        double descuento = 0.15;
        double total = precioProducto * (1.0 - descuento);

        System.out.println("Total a pagar: " + total);
    }
}
```

3. `boolean`: Solo admite `true` o `false` en minúsculas. Nada de números, nada de texto.

```
// Archivo: PruebaBooleanos.java
public class PruebaBooleanos {
    public static void main(String[] args) {
        boolean tieneAcceso = true; // ¡Estrictamente en minúscula!

        if (tieneAcceso) { // La condición va OBLIGATORIAMENTE entre
paréntesis
            System.out.println("Acceso concedido desde la terminal de
Java");
        }
    }
}
```

4. String: Un String en Java es en realidad un objeto que envuelve una secuencia de caracteres

```
// Archivo: PruebaStrings.java
public class PruebaStrings {
    public static void main(String[] args) {

        // Declaramos los objetos String (¡S mayúscula!)
        String usuarioGuardado = "admin";
    }
}
```

5. Integer: En Java las listas son serias y estructuradas, se llaman ArrayList y solo aceptan objetos. Por eso integer sustituye a int en las listas.

```
// Archivo: PruebaInteger.java
import java.util.ArrayList; // Necesitamos importar la herramienta de
listas

public class PruebaInteger {
    public static void main(String[] args) {
        // ERROR DE EXAMEN: ArrayList<int> lista = new ArrayList<>();
        <-- Esto NO COMPILA.

        // FORMA CORRECTA: Usamos la clase wrapper Integer
        ArrayList<Integer> notasExamen = new ArrayList<>();

        // Añadimos datos. Java convierte automáticamente el 'int' a
        'Integer' por detrás (Autoboxing)
        notasExamen.add(8);
        notasExamen.add(5);
        notasExamen.add(10);

        System.out.println("Notas guardadas en la lista: " +
        notasExamen);

        // Otro uso vital de Integer: Convertir un texto de la terminal
        a número para poder operar
        String textoDesdeTerminal = "25";
        int edad = Integer.parseInt(textoDesdeTerminal); // Convierte el
        String "25" en el int 25
        System.out.println("El año que viene tendrás: " + (edad + 1));
    }
}
```

Prioridades de operadores:

Java los bloques están muy claros. Aquí tenéis la tabla simplificada de prioridades de mayor a menor importancia:

1. Postfijos/Prefijos: ++, -- (¡Esto en Python ni existe! Incrementa o decrementa una variable directamente).
2. Multiplicativos: *, /, % (Multiplicación, división y el resto de la división).
3. Aditivos: +, - (Suma y resta).
4. Relacionales: <, >, <=, >=.
5. Igualdad: ==, !=.
6. Lógicos: && (Equivalente al and de Python) y luego || (Equivalente al or de Python).
7. Asignación: =, +=, -=, etc. (Los últimos de la fila).
8. Exponente (Potencias):

```
double base = 2;
double exponente = 3;
double resultado = Math.pow(base, exponente); // Resultado: 8.0

// Si necesitas el resultado en un entero (int), hay que hacer un
"casting":
int resultadoEntero = (int) Math.pow(base, exponente);
```

Sentencias condicionales:

Java usamos paréntesis obligatorios para las condiciones y llaves {} para abrir y cerrar los bloques de código.

1. La estructura if - else (y else if):

Cuándo se usa: Cuando tienes dos o más caminos posibles. Si la condición es verdadera se hace una cosa, y si es falsa (else), se hace otra.

```
// Archivo: CondicionalElse.java
public class CondicionalElse {
    public static void main(String[] args) {
        int nota = 4;

        if (nota >= 7) {
            System.out.println("Notable");
        } else if (nota >= 5) { // Equivalente al 'elif'
            System.out.println("Aprobado justo");
        } else {
            System.out.println("Suspenso");
        }
    }
}
```

2. La sentencia switch:

Cuando tienes una sola variable y quieres evaluar múltiples casos posibles de forma ordenada.

```
// Archivo: PruebaSwitch.java
public class PruebaSwitch {
    public static void main(String[] args) {
        int mes = 2;

        switch (mes) {
            case 1:
                System.out.println("Enero");
                break; // ¡VITAL! Si no pones 'break', el código sigue
                // ejecutando los siguientes casos.
            case 2:
                System.out.println("Febrero");
                break;
            case 3:
                System.out.println("Marzo");
                break;
            default: // Es el equivalente al 'else' final. Si no es ni
                // 1, ni 2, ni 3...
                System.out.println("Otro mes");
        }
    }
}
```

```
        break;
    }
}
```

Matrices:

En Java, una matriz (o array) es un contenedor indexado que guarda un conjunto de elementos del mismo y único tipo de dato. Además, tienen un tamaño fijo: una vez que lo creas, no se puede encoger ni estirar.

Java (Array rígido y seguro): `int[] misNumeros;` (Solo guardará enteros).

¿Cómo se crean (Declaración e Inicialización)?

1. Conociendo el tamaño pero no los valores iniciales

```
int[] notas = new int[5]; // Array listo para guardar 5 notas.
```

2. Conociendo los valores iniciales inmediatamente

```
int[] notas = {10, 8, 9};
```

¿Cómo añadir elementos a las matrices?

Aquí viene el gran choque cultural para un programador de Python: ¡En los arrays de Java NO existe el método `.append()`! No puedes meter más elementos de los que definiste al principio porque el tamaño en memoria es estático.

```
int[] numeros = new int[3];
numeros[0] = 14; // Añades en la primera posición
numeros[1] = 25; // Añades en la segunda posición
numeros[2] = 8;  // Añades en la terceraposición
// numeros[3] = 99; --> ¡ERROR! Lanza una excepción en tiempo de
ejecución
```

¿Cómo moverse (recorrer) y acceder a los elementos?

Al igual que en Python, los índices empiezan siempre en 0. Para ver cuántos elementos tiene, en Python usabas la función `len(lista)`. En Java, usamos la propiedad `.length`.

Acceso individual:

```
System.out.println(notas[0])
```

Bucleo para recorrerlo (Tradicional):

```
for (int i = 0; i < notas.length; i++) {  
    System.out.println(notas[i]);  
}
```

Matrices Bidimensionales (Tablas)

Una matriz bidimensional es simplemente un "array de arrays". Piensa en ella como una cuadrícula o tabla con filas y columnas.

Crearla vacía definiendo el tamaño (Filas x Columnas)

```
int[][] miTabla = new int[3][4];
```

Crearla inicializando los datos de golpe

```
int[][] matriz = {  
    {10, 20, 30},  
    {40, 50, 60}  
}; // ¡No te olvides del punto y coma final!
```

¿Cómo moverse (recorrer) en las matrices bidimensionales?

Para movernos por la cuadrícula necesitamos dos bucles for anidados: el primero controlará la fila en la que estamos parados y el segundo recorrerá las columnas de esa fila.

Para saber el tamaño, usamos la propiedad `.length`:

- `matriz.length` nos da el número total de filas.
- `matriz[i].length` nos da el número de columnas de esa fila en específico.

Código para recorrer e imprimir la matriz:

```
for (int i = 0; i < matriz.length; i++) {           // Recorre filas
    for (int j = 0; j < matriz[i].length; j++) { // Recorre columnas
        System.out.print(matriz[i][j] + " ");
    }
    System.out.println(); // Salto de línea al terminar cada fila
}
```

¿Cómo añadir o modificar elementos a Matrices Bidimensionales?

Al igual que con las matrices simples, el tamaño de una matriz bidimensional en Java es fijo. No existe un `.append()` para meter una fila nueva de la nada. Lo que hacemos es asignar valores directamente a las coordenadas existentes.

```
matriz[0][1] = 99;
```

Bucle for-each:

¡Excelente! El bucle for-each (que en Java formalmente se conoce como el bucle for mejorado o enhanced for) es lo más parecido que vas a encontrar al cómodo **for elemento in lista**: de Python. Dentro del paréntesis declaras una variable del mismo tipo de dato que almacena el array (en este caso `int nota`), pones dos puntos (`:`) que significan "dentro de", y luego el nombre de tu array (`notas`).

Matrices Unidimensionales:

Imagina que tenemos un array con notas de exámenes. Queremos leer cada nota.

```
int[] notas = {10, 8, 9};

for (int nota : notas) {
    System.out.println(nota);
}
```

Matrices Bidimensionales:

```
int[][] matriz = {
    {1, 2},
    {3, 4}
};
```

```
// 1. El primer bucle extrae cada FILA (que es un array int[])
for (int[] fila : matriz) {

    // 2. El segundo bucle extrae cada ELEMENTO (int) de esa fila
    for (int elemento : fila) {
        System.out.print(elemento + " ");
    }
    System.out.println(); // Salto de línea al terminar la fila
}
```

Metodos estaticos Arrays

Ordenar Arrays:

Java nos proporciona una clase de utilidad llamada `java.util.Arrays` que viene con métodos estáticos listos para usar. Uno de ellos es `sort()`.

```
import java.util.Arrays;
public class Listas {
    public static void main(String[] args) {
        int[] edade = {25, 27, 28, 22, 24, 29};
        // Ordena la tabla
        Listas.sort(edade);
        // Mostrar resultado
        for (int e : edade) {
            System.out.println(e);
        }
    }
}
```


Buscar sin binarySearch en Arrays:

Esta es la forma tradicional. Consiste en abrir un bucle e ir mirando uno a uno todos los elementos del array desde el principio hasta el final hasta encontrar el que buscas. Es el único método que funciona si el array está desordenado.

```
//siempre usar import java.util.Arrays; para los emtodos estaticos
import java.util.Arrays;
public class Listas {
    public static void main(String[] args) {
        int[] edade = {25, 27, 28, 22, 24, 29};
        // Ordena la tabla
        Arrays.sort(edade);
        // Mostrar resultado
        //Agora edade = {22, 24, 25, 27, 28, 29};
        int encontrar= 28;
        int i =0;
        boolean encontrado = false;
        for (;i<edade.length; i++) {
            if (edade[i]== encontrar){
                encontrado = true;
                break;
            }
        }
        if (encontrado){
            System.out.println("Esta en posición: " + i);
        }
        else{
            System.out.println("Non foi encontrado: -1");
        }
    }
}
```

Buscar con `binarySearch` en Arrays:

La búsqueda binaria es infinitamente más rápida que la secuencial, pero tiene una regla de oro obligatoria: el array TIENE QUE ESTAR ORDENADO previamente. Si no lo está, te devolverá basura.

Funciona dividiendo el array por la mitad repetidamente (como cuando buscas una palabra en un diccionario físico). Java ya tiene este algoritmo programado dentro de la clase de utilidad `java.util.Arrays`.

```
//siempre usar import java.util.Arrays; para los emtodos estaticos
import java.util.Arrays;
public class Listas {
    public static void main(String[] args) {
        int[] edade = {25, 27, 28, 22, 24, 29};
        // Ordena la tabla
        Arrays.sort(edade);
        // Mostrar resultado
        //Agora edade = {22, 24, 25, 27, 28, 29};

        int pos = Arrays.binarySearch(edade,28);
        if (pos>=0){
            System.out.println("Encontrado en posición: " + pos);
        }
        else{
            System.out.println("No encontrado");
        }
    }
}
```

Copiar SIN método estático :

Esta es la forma artesanal y la que demuestra que entiendes qué pasa por dentro. Consiste en crear un nuevo array vacío del mismo tamaño que el original, y luego abrir un bucle for para ir clonando los valores casilla por casilla.

```
//creamos un array del mismo tamaño
int edade2[] = new int[edade.length];
//recordemos la primera lista para pasar la posición a la nueva lista
for (int j=0; j<edade.length; j++){
    edade2[j] = edade[j];
}
```

Copiar CON método estático:

Cuando necesitas rendimiento y escribir menos código, Java pone a tu disposición un método estático súper optimizado dentro de la clase del sistema llamado `System.arraycopy()`.

```
//Copia toda la lista
int edade3[] = Arrays.copyOf(edade, edade.length);
//Copia por rango
int edade4[] = Arrays.copyOfRange(edade, 1, 4);
```

Comparar contenido:

Para no tener que escribir esa función en cada proyecto, Java nos ofrece el método estático `Arrays.equals()`. Este método ya tiene programada exactamente la misma lógica que acabamos de hacer a mano: compara tamaños y luego contenidos de forma secuencial.

```
import java.util.Arrays; // ¡Que no se te olvide importar la
herramienta!
int[] array1 = {1, 2, 3};
int[] array2 = {1, 2, 3};
int[] array3 = {1, 2, 4};
boolean caso1 = Arrays.equals(array1, array2); // Devolverá true
boolean caso2 = Arrays.equals(array1, array3); // Devolverá false (el
último elemento cambia)
```

EJEMPLO COMPLEJO REAL:

Crear a función que reciba como parámetro unha táboa unidimensional e devolte outra táboa cos valores da primeira, pero na que non haxa repeticións nos valores.

```
//siempre usar import java.util.Arrays; para los emtodos estaticos
import java.util.Arrays;
public class Listas {
    public static void main(String[] args) {
        //Creo la tabla
        int [] numeros ={ 1,2,4,9,8,2,4,7,2,5};
        //Ordeno
        Arrays.sort(numeros);
        //Creo el array vacio;
        int numeros2[] = new int[numeros.length];
        int posicion = 0;
        // Guardo el primer numero
        numeros2[posicion] = numeros[0];
        //Recorro la primera lista desde la posicion 1
        for (int i=1; i< numeros.length;i++){
            //comparo si un numero es distinto a su anterior;
            if (numeros[i] != numeros[i - 1]){
                //añado una posición a la lista 2
                posicion++;
                // Lo añadido a la lista
                numeros2[posicion]=numeros[i];
            }
        }
        //Copia para quitar los 0
        int numeros3[]= Arrays.copyOfRange(numeros2,0,7);
        //Recorrer la tabla nueva sin los 0
        for (int j=0; j<numeros3.length;j++){
            System.out.println(numeros3[j]);
        }
    }
}
```

String:

Un String es una secuencia de caracteres (letras, números, espacios en blanco y caracteres especiales como ?, @, etc.). Por ejemplo, "Hola Mundo" o "user123" son cadenas de texto. La gran diferencia con Python es que en Java los Strings no son un tipo de dato primitivo (como sí lo son int, double o boolean). Los string son inmutables

Declaración:

1. Asignación en la misma línea

```
String nombre = "Alejandro";  
String saludo = "¡Hola, mundo!";  
String vacio = ""; // Una cadena válida, pero vacía
```

2. Declarar primero y asignar el valor después

A veces no sabes qué texto va a contener la variable desde el principio (por ejemplo, porque estás esperando a que el usuario lo escriba por teclado). En ese caso, puedes separarlo en dos pasos:

```
// 1. Declaras la variable (se crea el espacio en la memoria)  
String mensaje;  
  
// 2. Le asignas el valor más adelante en tu código  
mensaje = "Bienvenidos a la clase de Java";
```

Longitud de una cadena:

Cuando estás trabajando con cadenas de texto (String), length() es un método (una función interna de la clase). Por lo tanto, es obligatorio ponerle paréntesis al final (). Devuelve el número total de caracteres que tiene la cadena (contando letras, números, símbolos y espacios en blanco).

```
String texto = "Hola Mundo";  
int tamanoTexto = texto.length(); // Devuelve 10 (lleva paréntesis)  
System.out.println(tamanoTexto);
```

Concatenación:

1. El operador + (El más utilizado)

Es la forma directa y la que heredamos mentalmente de Python. Puedes unir variables String, literales de texto e incluso mezclar texto con números u otros tipos de datos (Java convertirá automáticamente los números a texto por detrás).

```
String nombre = "Ana";
String apellido = "García";

// Concatenación simple usando el operador +
String nombreCompleto = nombre + " " + apellido;
System.out.println(nombreCompleto); // Imprime: Ana García
```

2. El método de instancia .concat()

Es un método que pertenece a cada objeto String. A diferencia de un método estático (que se llama usando el nombre de la clase), este se invoca directamente desde una variable de texto. Su única limitación es que solo acepta otro String como parámetro.

```
String saludo = "Hola ";
String destino = "Mundo";

// Invocamos el método desde la variable 'saludo'
String resultado = saludo.concat(destino);
System.out.println(resultado); // Imprime: Hola Mundo
```

Concatenación de datos no String:

Si usas el operador + y al menos uno de los operandos es un String, Java convertirá automáticamente el otro dato a texto antes de pegarlos.

```
public static void main(String args[]) {
    String novaCadea = "Aprendendo Java" + 17;
    System.out.println(novaCadea); //Aprendendo Java 17
    System.out.println("Total : " + 17 + 17); //Total: 1717
    System.out.println("Total : " + (17 + 17)); //Total: 34
    String numCadea = "17" + 17;
    System.out.println(numCadea); //1717
} //Fin método main
```

Acceso a cada caracteres:

Sin embargo, a diferencia de Python o de los propios arrays de Java, no puedes usar los corchetes [] para acceder a una letra; si escribes texto[0], el compilador dará un error inmediato.

Extraer un carácter específico: El método .charAt(índice)

Para recuperar el carácter que se encuentra en una posición determinada, debes utilizar obligatoriamente el método .charAt(int index). Al igual que en los arrays, los índices en un String son de base cero; es decir, el primer carácter está en la posición 0 y el último se encuentra en la posición longitud - 1.

```
String mensaje = "Java";

// Guardamos el primer carácter (índice 0) en una variable de tipo char
char primeraLetra = mensaje.charAt(0); // Devuelve 'J'
char terceraLetra = mensaje.charAt(2); // Devuelve 'v'
// Para la última letra del String
char ultimaLetra = mensaje.charAt(mensaje.length() - 1);

System.out.println("La primera letra es: " + primeraLetra);
```

Recorrer todos los caracteres (Bucle secuencial)

Es muy habitual en los exámenes que te pidan procesar una cadena letra a letra (por ejemplo, para contar cuántas veces aparece una vocal, o para verificar si hay números en el texto). Para hacer esto, combinamos un bucle for tradicional con los métodos .length() y .charAt()

```
String frase = "Aprendiendo programación en Java";
int contador = 0;

// El bucle va desde 0 hasta el tamaño de la cadena menos 1
for (int i = 0; i < frase.length(); i++) {
    char letraActual = frase.charAt(i); // Accedemos al carácter de la
    posición i

    // Comparamos el char (los char usan comillas simples y operador ==)
    if (letraActual == 'a' || letraActual == 'A') {
        contador++;
    }
}
```

```
System.out.println("La letra 'a' aparece " + contador + " veces.");
```

Posición: `indexOf()`:

El método `indexOf()` es una herramienta fundamental de la clase `String` que sirve para buscar cosas dentro de un texto. En lugar de devolverte el texto encontrado, te dice la posición (el índice) de la primera vez que aparece lo que estás buscando.

```
String texto = "Programación en Java";

// 1. Buscar la posición de un solo carácter (char usa comillas simples)
int posicionP = texto.indexOf('P'); // Devuelve 0 (es la primera letra)
int posicionM = texto.indexOf('m'); // Devuelve 6

// 2. Buscar la posición donde empieza una palabra entera
int posicionJava = texto.indexOf("Java"); // Devuelve 16 (la 'J' está en el índice 16)
```

si no encuentra nada:

```
String texto = "Hola Mundo";
int posicionX = texto.indexOf('X');

System.out.println(posicionX); // Imprime: -1
```

Buscar a partir de una posición concreta (Sobrecarga):

A veces no quieres saber dónde está la primera vez que aparece una letra, sino las siguientes. Para ello, `indexOf()` permite pasarle un segundo parámetro: el índice desde el cual debe empezar a buscar hacia la derecha.

```
//          0123456789012
String club = "Capitán Cacao";

// Busca la primera 'a' empezando desde el principio (índice 0)
int primeraA = club.indexOf('a'); // Devuelve 1

// Busca la siguiente 'a', pero le decimos que empiece a mirar desde el índice 2
int segundaA = club.indexOf('a', 2); // Devuelve 9
```


Métodos:

Recortar cadenas `.substring(int desde, int ata)`

Sirve para extraer un trozo (una subcadena) de un texto más grande. Tiene dos variantes (sobrecargas) muy importantes:

```
String palabra = "Anatomía";

String trozo1 = palabra.substring(3);    // Devuelve "tomía" (del
índice 3 al final)
String trozo2 = palabra.substring(0, 3); // Devuelve "Ana" (coge los
índices 0, 1 y 2. El 3 se excluye)
```

Reemplazar caracteres o trozos `.replace(char viejo, char nuevo )`

Busca todas las apariciones de un carácter (o de una frase) y las cambia por otra cosa.

```
String texto = "casa bonita";
String cambiado = texto.replace('a', 'e'); // Cambia todas las 'a' por
'es'
System.out.println(cambiado); // Imprime: cese bonite
```

Ordenar alfabéticamente `.compareTo(String outro)`

Este método no te devuelve un booleano, te devuelve un número entero (int). Sirve para saber qué palabra va antes en el diccionario (orden lexicográfico).

- Devuelve 0: Si ambas cadenas son exactamente iguales.
- Devuelve un número NEGATIVO (< 0): Si la cadena que llama al método va antes en el diccionario que la que pasas por parámetro.
- Devuelve un número POSITIVO (> 0): Si la cadena que llama al método va después en el diccionario

```
int comp1 = "casa".compareTo("coche"); // Devuelve un número NEGATIVO
("casa" va antes)
int comp2 = "tren".compareTo("tren");   // Devuelve 0 (son idénticas)
```

Cambiar a Mayúsculas / Minúsculas .toUpperCase() y .toLowerCase()

Transforman todo el texto a letras mayúsculas o minúsculas. Son vitales para "normalizar" textos antes de compararlos (así evitas que el programa falle si el usuario escribe "Juan" o "juan").

```
String original = "Java360";
System.out.println(original.toUpperCase()); // Imprime: JAVA360
System.out.println(original.toLowerCase()); // Imprime: java360
```

Limpiar espacios en blanco .trim()

Elimina de forma automática los espacios en blanco que se hayan quedado sueltos únicamente al principio y al final de la cadena. No toca los espacios intermedios.

```
String entradaUsuario = "  mi password secreto  ";
String limpio = entradaUsuario.trim();
System.out.println(limpio); // Imprime: "mi password secreto"
```

Romper el texto en trozos (Arrays) String[] split(String regex)

Este es el método más potente cuando te dan un texto largo y necesitas trocearlo. Le pasas un "separador" (expresión regular) y el método te devuelve un array de Strings (String[]) donde cada casilla es un pedazo del texto original.

```
String compra = "Patatas,Huevos,Leche,Pan";

// Rompemos la cadena cada vez que encuentre una coma
String[] productos = compra.split(",");

// Ahora tenemos un array tradicional que podemos recorrer
System.out.println(productos[0]); // Imprime: Patatas
System.out.println(productos[2]); // Imprime: Leche
System.out.println("Total productos: " + productos.length); // Imprime:
4 (Ojo: length sin paréntesis porque es un array)
```

OBJETOS:

En Java todo tiene un lugar, un tipo y una razón de ser; aquí no dependemos de si pusiste un espacio de más o de menos en la indentación.

En Java, necesitas definir de forma explícita y estricta tres partes esenciales:

1. Atributos (Variables): Qué datos guarda el objeto (¡obligatorio decir su tipo de dato!).
2. Constructor: El método especial para crear el objeto.
3. Métodos (Funciones): Qué acciones puede realizar el objeto

```
public class Alumno {
    // 1. ATRIBUTOS: Hay que especificar el tipo obligatoriamente
    (String, int, etc.)
    private String nombre;
    private double nota;

    // 2. CONSTRUCTOR: Lleva exactamente el mismo nombre de la clase
    (¡nada de __init__!)
    public Alumno(String nombre, double nota) {
        this.nombre = nombre; // "this" hace el mismo trabajo que el
        "self" de Python
        this.nota = nota;
    }

    // 3. MÉTODO: Debes especificar qué tipo de dato devuelve (o "void"
    si no devuelve nada)
    public void mostrarInfo() {
        System.out.println("Alumno: " + this.nombre + ", Nota: " +
        this.nota);
    }
}
```

En Java (Con Getters, Setters y control total):

En Java somos mucho más ordenados y precavidos. Usamos algo llamado encapsulación, que básicamente significa: "Nadie toca mis variables directamente desde fuera de la clase".

```
public class Alumno {
    // 1. Atributos PRIVADOS (Nadie puede hacer alumno.nota desde otra
    // clase)
    private String nombre;
    private double nota;

    // Constructor
    public Alumno(String nombre, double nota) {
        this.nombre = nombre;
        this.nota = nota;
    }

    // 2. EL GETTER: Sirve para LEER el valor. Su nombre empieza por
    // "get".
    // Debe retornar el mismo tipo de dato que la variable (String).
    public String getNombre() {
        return this.nombre;
    }

    // 3. EL SETTER: Sirve para MODIFICAR el valor. Su nombre empieza
    // por "set".
    // No retorna nada (void) y recibe el nuevo valor como parámetro.
    public void setNombre(String nuevoNombre) {
        this.nombre = nuevoNombre;
    }

    // OTRO GETTER (Para la nota)
    public double getNota() {
        return this.nota;
    }

    // OTRO SETTER CON CONTROL DE SEGURIDAD
    // Aquí ves la verdadera utilidad: ¡Podemos validar los datos!
    public void setNota(double nuevaNota) {
        if (nuevaNota >= 0 && nuevaNota <= 10) {
            this.nota = nuevaNota; // Solo se cambia si es una nota
            válida
        } else {
            System.out.println("Error: La nota debe estar entre 0 y
            10.");
        }
    }
}
```

```
}
```

Instanciar la clase

Somos bastante más explícitos y formales: necesitamos usar una palabra reservada mágica llamada `new` y, por supuesto, declarar el tipo de dato de la variable que va a guardar ese objeto.

```
// 1. Tipo de dato (Clase) -> 2. Nombre de la variable -> 3. Signo '='  
// 4. Palabra clave 'new' -> 5. Constructor con los parámetros -> 6.  
¡Punto y coma!  
Alumno unAlumno = new Alumno("Carlos", 8.5);
```

Métodos funcionais:

Este es un ejemplo de método funcional que compara la autonomía en km/l de cada vehículo pasado como parámetro; devuelve 0 si son iguales, 1 si el primer vehículo tiene mayor autonomía que el segundo y -1 si el segundo vehículo tiene mayor autonomía que el primero.

```
public int comparaA(Vehiculo v1, Vehiculo v2){  
    if(v1.getKmPorLitro() == v2.getKmPorLitro())  
        return 0;  
    if(v1.getKmPorLitro() > v2.getKmPorLitro())  
        return 1;  
    return -1;  
} //Fin método comparaA
```

Objeto como parámetro:

En Java, para lograr exactamente lo mismo, la sintaxis te obliga a especificar que el parámetro que recibe el método es un objeto de la clase `Coche`. Míralo aquí:

```
public class Coche {  
    private String marca;  
  
    // Construtor  
    public Coche(String marca) {  
        this.marca = marca;  
    }  
}
```

```

        // Getter para obtener la marca
        public String getMarca() {
            return this.marca;
        }
    }

    public class Principal {
        // Aquí está el truco: especificamos el Tipo (Coche) y luego el
        // nombre (unCoche)
        public static void mostrarMarca(Coche unCoche) {
            System.out.println("La marca del coche es: " +
unCoche.getMarca());
        }

        public static void main(String[] args) {
            Coche miAuto = new Coche("Toyota");
            mostrarMarca(miAuto); // Pasamos el objeto directamente
        }
    }
}

```

Variable estática:

```

public class Coche {
    private String modelo;          // Variable de instancia (cada coche
    // tiene el suyo)
    public static int totalCoches = 0; // ¡Variable estática! Compartida
    // por todos

    // Constructor
    public Coche(String modelo) {
        this.modelo = modelo;
        totalCoches++; // Cada vez que creamos un coche, sumamos al
        // contador global
    }
}

public class Principal {
    public static void main(String[] args) {
        Coche c1 = new Coche("Ibiza");
        Coche c2 = new Coche("Leon");

        // Para acceder, lo ideal es usar directamente el nombre de la
    }
}

```

```

Clase
    System.out.println("Total de coches: " + Coche.totalCoches); //
Imprime 2

    // Aunque se puede acceder desde un objeto, no se recomienda
porque confunde
    // System.out.println(c1.totalCoches); // También daría 2
}
}

```

Abstracto:

El propio lenguaje tiene la palabra reservada abstracta. Mira qué limpio y directo queda:

```

// Usamos 'abstract' para decirle a Java: "Ojo, esta clase es un molde
incompleto"
public abstract class Animal {
    private String nome;

    public Animal(String nome) {
        this.nome = nome;
    }

    // Un método abstracto NO tiene cuerpo (no lleva llaves {}), termina
en punto y coma.
    // Obliga a los hijos a escribir su propia versión.
    public abstract void facerSon();
}

// La clase hija usa 'extends' para heredar
public class Perro extends Animal {
    public Perro(String nome) {
        super(nome); // Llamamos al constructor del padre
    }

    // Implementamos el método obligatorio
    @Override
    public void facerSon() {
        System.out.println("Guauuu");
    }
}
}

```

En Java, el compilador es todavía más estricto. Ni siquiera te dejará compilar el proyecto si te olvidas de rellenar el método abstracto en la clase hija. Mira qué limpio queda:

```
// La clase abstracta: Define el molde global
public abstract class Empleado {
    private String nome;

    public Empleado(String nome) {
        this.nome = nome; // Usamos 'this' como vuestro 'self'
    }

    public String getNome() {
        return this.nome;
    }

    // Método abstracto: No tiene cuerpo, termina en ';'
    public abstract double calcularSalario();
}

// Clase concreta: "Hereda" de la abstracta usando 'extends'
public class EmpleadoContratado extends Empleado {
    private double sueldoFijo;

    public EmpleadoContratado(String nome, double sueldoFijo) {
        super(nome); // ¡OBLIGATORIO! Primero construimos la parte del
padre
        this.sueldoFijo = sueldoFijo;
    }

    // OBLIGATORIO: Cumplimos el contrato implementando el método
@Override
    public double calcularSalario() {
        return this.sueldoFijo;
    }
}
```


Interfaces:

Una interfaz es un contrato de comportamiento. No le importa quién eres, solo le importa qué sabes hacer. Si una clase firma (implementa) el contrato, está obligada a cumplirlo.

¿Para qué sirven y su Sintaxis básica?

Sirven para conectar clases que no tienen nada que ver entre sí, pero que comparten una acción. Por ejemplo, un Avion y un Pajaro no heredan de la misma madre, pero los dos saben volar.

```
public interface Volador {  
    // Por defecto, los métodos aquí son abstractos (no tienen cuerpo)  
    void volar();  
}
```

Implementación (Cómo se usa)

En Python simplemente programabas el método en ambas clases. En Java usas la palabra clave implements. El compilador vigilará que no te olvides de programar el método.

```
public class Avion implements Volador {  
    @Override  
    public void volar() {  
        System.out.println("Arrancando motores y despegando pista  
2...");  
    }  
}  
  
public class Pajaro implements Volador {  
    @Override  
    public void volar() {  
        System.out.println("Aleteando las alas hacia el sur...");  
    }  
}
```

Atributos en una Interfaz

¡Ojo aquí! Una interfaz no puede tener variables de instancia normales (no puedes guardar el "estado" de un objeto).

Cualquier variable que declares dentro de una interfaz será automáticamente public static final (es decir, una constante global).

```
public interface Volador {  
    int ALTURA_MAXIMA = 10000; // Es una constante fija, no se puede  
    cambiar  
}
```

Métodos por Defecto (default)

Antiguamente las interfaces solo tenían métodos vacíos. Pero, ¿qué pasa si añades un método nuevo a una interfaz que ya usan 100 clases? ¡Romperías todo el código!

Para evitarlo, Java introdujo los métodos default. Son métodos con cuerpo dentro de la interfaz. Si la clase hija quiere, los usa; si no, los ignora.

```
public interface Volador {  
    void volar();  
  
    // Método por defecto: las clases hijas lo heredan automáticamente  
    con esta lógica  
    default void aterrizar() {  
        System.out.println("Descendiendo de forma segura...");  
    }  
}
```

Métodos Estáticos (static)

Igual que vimos con las variables estáticas, un método static dentro de una interfaz le pertenece a la interfaz misma, no a las clases que la implementan. Sirve para funciones de utilidad.

```
public interface Volador {  
    static boolean puedeVolarConLluvia(int intensidad) {  
        return intensidad < 50;  
    }  
}  
// Se llama directamente así: Volador.puedeVolarConLluvia(30);
```

Métodos Privados (private)

A veces tienes varios métodos default o static en tu interfaz que repiten código. Para no duplicarlo, puedes crear métodos private internos que solo se pueden ver y usar dentro de la propia interfaz.

```
public interface Volador {
    default void despegar() {
        log("Iniciando despegue");
    }

    // Método privado auxiliar para no repetir código
    private void log(String mensaje) {
        System.out.println("[LOG VOLADOR]: " + mensaje);
    }
}
```

Herencia entre Interfaces

¡Cuidado! Una clase hace implements de una interfaz. Pero una interfaz hereda de otra interfaz usando extends. Además, a diferencia de las clases, ¡aquí sí se permite la herencia múltiple!.

```
public interface SuperHeroe extends Volador, Luchador {
    void salvarElMundo();
}
```

Declaración de variables tipo Interface (Polimorfismo)

```
public class Principal {
    public static void main(String[] args) {
        // Declaramos la variable de tipo interfaz
        Volador miObjeto;

        miObjeto = new Avion();
        miObjeto.volar(); // Ejecuta el vuelo del avión

        miObjeto = new Pajaro();
        miObjeto.volar(); // Ejecuta el vuelo del pájaro
    }
}
```

Clases Anónimas

Imagina que necesitas crear un objeto rápido que implemente una interfaz, pero te da pereza crear un archivo .java entero solo para esa tontería. Puedes crear una Clase Anónima: una clase sin nombre que se define y se instancia al mismo tiempo directamente en el código.

```
public class Principal {
    public static void main(String[] args) {
        // Creamos un volador "al vuelo" sin crear una clase formal
        Volador superMan = new Volador() {
            @Override
            public void volar() {
                System.out.println("Volando con capa a toda
velocidad!");
            }
        }; // ¡Ojo al punto y coma al final de la llave!

        superMan.volar();
    }
}
```

Copare y comparable

Por defecto, si tienes una lista de números o de textos, Java sabe cómo ordenarla (ArrayList ordena los números de menor a mayor y los textos alfabéticamente). Pero... ¿qué pasa si tienes una lista de objetos tuyos, como Alumno o Producto? Java no sabe si quieres ordenarlos por nota, por edad, por nombre o por precio.

Interfaces con tipos genéricos

Una interfaz genérica no define un tipo de datos fijo. En su lugar, utiliza una letra comodín (normalmente T de Type, E de Element o K, V de Key, Value). Quien implemente la interfaz decidirá más tarde con qué clase real quiere trabajar.

Para que lo entiendas de cara al examen, vamos a diseñar una interfaz genérica para un Contenedor de Datos y verás las dos formas que existen de implementarla

Definir la Interfaz Genérica

Creemos la interfaz Contenedor<T>. Fíjate que donde antes ponías un tipo fijo (como Socio o String), ahora ponemos T:

```
// El <T> indica que esta interfaz se adaptará a cualquier objeto
public interface Contenedor<T> {

    void guardar(T elemento); // Recibe un objeto de tipo T

    T recuperar(int posicion); // Devuelve un objeto de tipo T
}
```

Implementar la Interfaz

Cuando una clase quiere implementar esta interfaz (implements Contenedor), tiene dos opciones en Java:

Es cuando la clase sabe exactamente con qué va a trabajar. Por ejemplo, creamos un contenedor exclusivo de objetos Socio. Al hacer esto, todos los métodos con T se transforman automáticamente en Socio.

```
import java.util.ArrayList;
import java.util.List;

// Indicamos que esta clase implementa la interfaz FIJANDO el tipo a
<Socio>
public class AlmacenSocios implements Contenedor<Socio> {

    private List<Socio> lista = new ArrayList<>();

    // Java te obliga a usar Socio donde antes había una T
    @Override
    public void guardar(Socio socio) {
        lista.add(socio);
    }

    @Override
    public Socio recuperar(int posicion) {
        return lista.get(posicion);
    }
}
```

La Interfaz Comparable (El orden natural)

Se utiliza cuando una clase tiene un orden por defecto u orden natural (por ejemplo, lo más lógico es que los alumnos se ordenen siempre por su ID o por su nota).

- Regra clave: La propia clase que quieres ordenar debe implementar Comparable<MiClase>.
- Método obligatorio: Te obliga a programar el método compareTo(Objeto o).
- Cómo funciona el método: Compara el objeto actual (this) con el que le llega por parámetro (o). Debe devolver:
 - Un número negativo si this va antes que o.
 - Un cero si son exactamente iguales.
 - Un número positivo si this va después que o.

ejemplo

```
// Implementamos Comparable e indicamos el tipo entre picos <Alumno>
public class Alumno implements Comparable<Alumno> {
    private String nome;
    private double nota;

    public Alumno(String nome, double nota) {
        this.nome = nome;
        this.nota = nota;
    }

    // OBLIGATORIO: Programar el criterio de ordenación natural
    @Override
    public int compareTo(Alumno outro) {
        // Queremos ordenar de mayor a menor nota
        if (this.nota > outro.nota) {
            return -1; // Devuelve negativo -> 'this' va primero
        } else if (this.nota < outro.nota) {
            return 1; // Devuelve positivo -> 'this' va detrás
        } else {
            return 0; // Son iguales
        }
    }

    @Override
    public String toString() {
        return nome + " (" + nota + ")";
    }
}
```

Cómo se usa en el ejecutable:

```
import java.util.ArrayList;
import java.util.Collections;

public class PrincipalComparable {
    public static void main(String[] args) {
        ArrayList<Alumno> clase = new ArrayList<>();
        clase.add(new Alumno("Antón", 5.5));
        clase.add(new Alumno("Brais", 9.0));
        clase.add(new Alumno("Carlos", 7.2));

        // ¡Magia! Collections.sort() mira automáticamente el método
        // compareTo del Alumno
        Collections.sort(clase);

        System.out.println("Alumnos ordenados por nota (Orden
Natural):");
        System.out.println(clase); // Imprime: [Brais (9.0), Carlos
(7.2), Antón (5.5)]
    }
}
```

La Interfaz Comparator (Órdenes alternativos)

¿Qué pasa si el orden natural de los alumnos es por nota, pero en una sección de nuestra aplicación los queremos ordenar alfabéticamente por su nombre? No podemos duplicar el método compareTo.

Para eso se usa Comparator. Creamos una clase "juez" externa que solo sirve para comparar.

- Regra clave: Se crea una clase independiente que implementa Comparator<MiClase>.
- Método obligatorio: Te obliga a programar el método compare(Objeto o1, Objeto o2).
- Cómo funciona: Recibe dos objetos de golpe (o1 y o2) y aplica la misma lógica de enteros (negativo, cero o positivo).

Ejemplo:

```
import java.util.Comparator;

// Clase externa que actúa como juez para ordenar Alumnos por Nombre
public class ComparadorPorNombre implements Comparator<Alumno> {

    @Override
    public int compare(Alumno a1, Alumno a2) {
        // Truco de examen: Como la clase String ya tiene su propio
        compareTo,
        // lo aprovechamos para ordenar alfabéticamente
        return a1.getNome().compareTo(a2.getNome());
    }
}
```

Cómo se usa en el ejecutable:

```
import java.util.ArrayList;
import java.util.Collections;

public class PrincipalComparator {
    public static void main(String[] args) {
        ArrayList<Alumno> clase = new ArrayList<>();
        clase.add(new Alumno("Carlos", 7.2));
        clase.add(new Alumno("Antón", 5.5));
        clase.add(new Alumno("Brais", 9.0));

        // Le pasamos a Collections.sort() la lista Y NUESTRO JUEZ
        EXTERNO
        Collections.sort(clase, new ComparadorPorNombre());

        System.out.println("Alumnos ordenados alfabéticamente (Orden
        Alternativo):");
        System.out.println(clase); // Imprime: [Antón (5.5), Brais
        (9.0), Carlos (7.2)]
    }
}
```

EJERCICIO EJEMPLO INTERFACES RESUELTO

Imagina que estamos desarrollando un software para una tienda de electrónica. Queremos gestionar diferentes tipos de productos, pero nos interesa aislar un comportamiento específico: los productos que son recargables (como teléfonos móviles o auriculares inalámbricos).

1. Crea una interfaz llamada Recargable.
2. Crea una clase llamada Telefono que implemente esa interfaz.

```
// =====  
// 1. DEFINICIÓN DE LA INTERFAZ (EL CONTRATO)  
// =====  
// Usamos la palabra clave 'interface'.  
// Recuerda que no define "qué es" el objeto, sino "qué sabe hacer".  
public interface Recargable {  
  
    // Atributo en interfaz: Automáticamente es 'public static final'  
    (una constante).  
    // No puede cambiar de valor. Representa el nivel máximo de carga.  
    int CARGA_MAXIMA = 100;  
  
    // Método abstracto: No tiene cuerpo (lleva ';').  
    // Obliga a las clases que implementen la interfaz a darle una  
    lógica.  
    void cargar(int porcentaje);  
  
    // Método por defecto (default): Tiene cuerpo.  
    // Las clases hijas lo heredan automáticamente, pero pueden  
    reescribirlo si quieren.  
    default void mostrarEstadoCarga(int cargaActual) {  
        System.out.println("Nivel de energía actual: " + cargaActual +  
"% de un máximo de " + CARGA_MAXIMA + "%.");  
    }  
}
```

```
// =====  
// 2. IMPLEMENTACIÓN EN UNA CLASE CONCRETA  
// =====  
// La clase 'Telefono' firma el contrato con 'Recargable' usando  
// 'implements'.  
public class Telefono implements Recargable {  
    // Atributos propios de la instancia de la clase Telefono  
    private String modelo;  
    private int bateriaActual;  
  
    // Constructor normal para inicializar el teléfono  
    public Telefono(String modelo) {  
        this.modelo = modelo;    // 'this' hace referencia a la  
variable de este objeto
```

```
        this.bateriaActual = 20;    // Todos los teléfonos nuevos vienen
con 20% de batería de fábrica
    }

    // OBLIGATORIO: Implementamos el método abstracto de la interfaz.
    // Usamos @Override para indicarle al compilador que estamos
sobrescribiendo el método.
    // ¡Cuidado de no olvidarte de poner 'public'!
    @Override
    public void cargar(int porcentaje) {
        this.bateriaActual += porcentaje;

        // Controlamos que la carga no supere la constante de la
interfaz
        if (this.bateriaActual > CARGA_MAXIMA) {
            this.bateriaActual = CARGA_MAXIMA;
        }
        System.out.println("-> Cargando el " + modelo + " un +" +
porcentaje + "%...");
    }

    // Getter normal para que el mundo exterior pueda consultar la
batería actual
    public int getBateriaActual() {
        return this.bateriaActual;
    }
}
```

Tipos Genericos

¿Qué es un tipo genérico?

En Python las listas aceptan lo que les echas: puedes meter un texto, un número y un objeto tuyo en la misma lista. Python no se queja al programar... pero como metas la pata y luego intentes operar con ellos, el programa te explotará en la cara mientras el usuario lo está usando.

En Java, para evitar esos sustos, somos muy estrictos con los tipos. Un tipo genérico (o Generic) es simplemente un comodín (una etiqueta, que por convención suele ser la letra <T>) que le ponemos a una clase o método. Le dice a Java: "Oye, aquí va a ir un tipo de datos, pero ya te diré cuál cuando cree el objeto. Eso sí, una vez que te lo diga, asegúrate de que nadie meta un dato erróneo".

Ejemplo SIN tipo genérico vs CON tipo genérico

Código SIN Genéricos

```
import java.util.ArrayList;

public class EjemploSinGenericos {
    public static void main(String[] args) {
        // Creamos una lista "viva la virgen" (acepta cualquier Object)
        ArrayList lista = new ArrayList();

        lista.add("Hola Alumnos"); // Metemos un String
        lista.add(123);             // ¡Java nos deja meter un Integer!

        // El gran problema: Al recuperar los datos, estamos OBLIGADOS
        // a hacer un "casting" (conversión manual).
        String texto = (String) lista.get(0);
        System.out.println(texto);

        // ¡Peligro! Si intentamos recuperar el segundo elemento como
        String...
        // String textoEquivocado = (String) lista.get(1);
        // ☀ ¡El programa compila bien pero EXPLOTA al ejecutarse!
        (ClassCastException)
    }
}
```

Código CON Genéricos

```
import java.util.ArrayList;

public class EjemploConGenericos {
    public static void main(String[] args) {
        // Le decimos a Java: "Esta lista es EXCLUSIVAMENTE para
        Strings"
        ArrayList<String> lista = new ArrayList<>();

        lista.add("Hola Alumnos");

        // lista.add(123); ❌ ¡Error de compilación inmediato!
        // Java no te deja ni compilar el programa si intentas meter un
        número.

        // Ventaja: Al recuperar el dato, ya NO necesitas hacer castings
        raros

        String texto = lista.get(0);
        System.out.println(texto);
    }
}
```

¿Qué es una clase genérica y Ejemplo?

Una clase genérica es una clase normal y corriente, pero que está parametrizada con ese comodín que te comenté (<T>). Funciona como una plantilla o un molde para almacenar o procesar datos sin importar el tipo concreto.

Pensemos en una Caja. Una caja sirve para guardar cosas. Puede ser una caja de herramientas, una caja de juguetes o una caja de zapatos. En lugar de programar tres clases distintas, programamos una sola clase genérica:

```
// El <T> indica que es una clase genérica. 'T' significa 'Type'
public class Caja<T> {
    private T contenido; // El atributo se adaptará al tipo de dato
    elegido

    public Caja(T contenido) {
        this.contenido = contenido;
    }

    public T getContenido() {
        return this.contenido;
    }
}
```

```
}  
}
```

Uso de múltiples parámetros genéricos

Puedes usar los que te hagan falta separándolos por comas (por convención se usan letras como T para el tipo principal, E para elementos de listas, K para llaves y V para valores).

Imagina que queremos guardar un Par de datos que están relacionados, como un ID de usuario (número) y su Nombre (texto):

```
// Usamos dos comodines independientes: <K, V>  
public class Par<K, V> {  
    private K clave;  
    private V valor;  
  
    public Par(K clave, V valor) {  
        this.clave = clave;  
        this.valor = valor;  
    }  
  
    public void mostrarPar() {  
        System.out.println("Clave: " + clave + " | Valor: " + valor);  
    }  
}  
  
// En el main lo usaríamos así:  
// Par<Integer, String> alumno = new Par<>(1, "Antón");
```

Métodos genéricos

A veces no necesitas que toda la clase sea genérica, solo te interesa que un método en específico sea capaz de aceptar diferentes tipos de datos.

Para declarar un método genérico, ponemos el comodín <T> justo antes del tipo de retorno del método. Mira este método estático para mostrar cualquier array por pantalla:

```

public class Utilidad {

    // Método genérico estático que acepta un array de cualquier tipo de
    // objeto T
    public static <T> void mostrarArray(T[] array) {
        for (T elemento : array) {
            System.out.print(elemento + " ");
        }
        System.out.println();
    }
}

// En el main, el mismo método te sirve para todo:
// String[] nombres = {"Ana", "Luis"};
// Integer[] notas = {8, 9, 5};
// Utilidad.mostrarArray(nombres); // Funciona!
// Utilidad.mostrarArray(notas);   // También funciona!

```

Restricción de tipos genéricos

Para solucionar esto, podemos restringir el comodín usando la palabra clave `extends` seguido de la clase límite. Le decimos a Java: "Acepto cualquier tipo T, siempre y cuando sea hijo (o implemente) a este grupo".

```

public class CalculadoraEspecial {

    // Restricción: 'T' obligatoriamente tiene que ser un número
    // (Integer, Double, Float...)
    public static <T extends Number> double sumar(T a, T b) {
        // Convertimos los tipos genéricos a double para poder sumarlos
        // de forma segura
        return a.doubleValue() + b.doubleValue();
    }
}

// En el main:
// CalculadoraEspecial.sumar(10, 20.5);      // ¡Perfecto! Cruza un
// Integer y un Double
// CalculadoraEspecial.sumar("10", "20");   // ❌ ¡Error de compilación!
// Un String no es un Number

```

Colecciones Java

¿Qué son las Colecciones en Java y la Estructura del JCF?

Olvídate de los arrays tradicionales de tamaño fijo que vimos en el Tema 2. Las colecciones son estructuras de datos dinámicas que pueden crecer y encogerse en memoria a tu antojo a medida que añades o quitas elementos.

El JCF es simplemente un conjunto de clases e interfaces ya programadas en Java para facilitarnos la vida. Su estructura (simplificada para examen) se organiza así:

- Collection (La interfaz madre): Define el comportamiento básico de cualquier saco de datos (añadir, borrar, buscar...).
- List (Interfaz hija): Admite elementos duplicados y mantiene un orden estricto por posición (índices). El rey aquí es ArrayList.
- Set (Interfaz hija): NO admite duplicados y el orden no importa. El rey aquí es HashSet.
- Map (Interfaz independiente): Estructura de Clave-Valor (como los diccionarios de Python). El rey es HashMap.

Ejemplo de List (Uso de ArrayList)

¿Cuándo se usa? Cuando el orden en el que metes los datos importa (quieres saber quién va primero, quién va segundo) y además permites que se repitan elementos

```
import java.util.ArrayList;
import java.util.List;

public class EjemploList {
    public static void main(String[] args) {
        // Polimorfismo: Declaramos la interfaz List, instanciamos
        ArrayList
        List<String> colaImpresion = new ArrayList<>();

        // Añadimos elementos en orden
        colaImpresion.add("Informe_Final.pdf");
        colaImpresion.add("Foto_Equipo.png");
        colaImpresion.add("Informe_Final.pdf"); // ¡Permite duplicados!

        System.out.println("=== COLA DE IMPRESIÓN (LIST) ===");
        // Al recorrerlo, mantiene estrictamente el orden de llegada
        for (String documento : colaImpresion) {
            System.out.println("Imprimiendo: " + documento);
        }

        // Podemos acceder por posición exacta como hacíamos en el Tema
```

```

        System.out.println("El primer documento a imprimir es: " +
colaImpresion.get(0));
    }
}

```

Ejemplo de Set (Uso de HashSet)

Cuándo se usa? Cuando lo único que te importa es saber si algo pertenece o no al grupo. No se permiten duplicados bajo ningún concepto y el orden de los elementos da exactamente igual.

```

import java.util.HashSet;
import java.util.Set;

public class EjemploSet {
    public static void main(String[] args) {
        // Polimorfismo: Declaramos la interfaz Set, instanciamos
        HashSet
        Set<String> alumnosAsistentes = new HashSet<>();

        // Registramos las entradas de los alumnos
        alumnosAsistentes.add("Antón");
        alumnosAsistentes.add("Brais");
        alumnosAsistentes.add("Antón"); // ¡Intento duplicar a Antón!
        Java lo ignorará en silencio
        alumnosAsistentes.add("Carlos");

        System.out.println("\n=== ALUMNOS ASISTENTES (SET) ===");
        // Al recorrerlo, verás que Antón solo sale una vez y el orden
        puede haber cambiado
        for (String alumno : alumnosAsistentes) {
            System.out.println("Asistente verificado: " + alumno);
        }

        // Búsqueda ultra rápida: ideal para saber si alguien vino
        if (alumnosAsistentes.contains("Brais")) {
            System.out.println("Efectivamente, Brais está en el aula.");
        }
    }
}

```


Ejemplo de Map (Uso de HashMap)

¿Cuándo se usa? Este va por libre. No hereda de la interfaz Collection. Se usa para asociar una Clave única a un Valor (es el equivalente exacto a los diccionarios que tanto usabais en Python).

```
import java.util.HashMap;
import java.util.Map;

public class EjemploMap {
    public static void main(String[] args) {
        // Declaramos el Map indicando que la Clave es String y el Valor
        // es Integer (Wrapper)
        Map<String, Integer> agenda = new HashMap<>();

        // Para añadir datos no usamos .add(), usamos .put(clave, valor)
        agenda.put("Antón", 600111222);
        agenda.put("Brais", 655333444);
        agenda.put("Carlos", 699555666);

        // ¡Ojo! Si repites una clave, machaca el valor anterior (lo
        // actualiza)
        agenda.put("Antón", 600999999);

        System.out.println("\n=== AGENDA TELEFÓNICA (MAP) ===");
        // Buscar un dato es inmediato pasándole su clave con
        // .get(clave)
        int telefonoDeBrais = agenda.get("Brais");
        System.out.println("El teléfono de Brais es: " +
            telefonoDeBrais);

        // Si quieres recorrer un Map entero en un bucle for-each, se
        // hace así:
        for (Map.Entry<String, Integer> entrada : agenda.entrySet()) {
            System.out.println("Nombre: " + entrada.getKey() + " ->
            Teléfono: " + entrada.getValue());
        }
    }
}
```

La Interfaz Collection y sus Métodos Principales

Cualquier clase que pertenezca al JCF (salvo los Maps) hereda de la interfaz Collection, por lo que todos estos métodos te van a funcionar igual en un ArrayList o en un HashSet:

- `add(E e)`: Añade un elemento.
- `remove(Object o)`: Elimina un elemento concreto.
- `size()`: Te dice cuántos elementos hay dentro.
- `isEmpty()`: Devuelve true si la colección está vacía.
- `contains(Object o)`: Busca si un elemento está dentro (devuelve true/false).
- `clear()`: Vacía la colección por completo.

Ejemplo con arraylist

```
import java.util.ArrayList;
import java.util.Collection;

public class EjemploCollectionList {
    public static void main(String[] args) {

        // 1. Creamos la colección usando Polimorfismo.
        // Tipo interfaz (Collection) e instanciamos la clase concreta
        // (ArrayList).
        Collection<String> playlist = new ArrayList<>();

        // 2. Método isEmpty() - Comprobamos si la lista está vacía al
        // empezar
        System.out.println("¿La playlist está vacía?: " +
            playlist.isEmpty()); // Imprime: true

        // 3. Método add(E e) - Añadimos elementos a la colección
        playlist.add("Bohemian Rhapsody");
        playlist.add("Thriller");
        playlist.add("Hotel California");
        playlist.add("Thriller"); // Al ser un ArrayList, permite
        // elementos duplicados

        System.out.println("\n--- Playlist creada ---");

        // 4. Método size() - Averiguamos cuántos elementos se han
        // guardado
        System.out.println("Número de canciones en la lista: " +
            playlist.size()); // Imprime: 4
    }
}
```

```

        // 5. Método contains(Object o) - Buscamos si existe una canción
concreta
        boolean tieneThriller = playlist.contains("Thriller");
        System.out.println("¿Está la canción 'Thriller' en la lista?: "
+ tieneThriller); // Imprime: true
        System.out.println("¿Está 'Despacito'?: " +
playlist.contains("Despacito")); // Imprime: false

        // 6. Método remove(Object o) - Eliminamos un elemento en
concreto
        // Ojo: En un ArrayList, remove elimina la primera coincidencia
que encuentra
        playlist.remove("Thriller");
        System.out.println("\nEliminando una copia de 'Thriller'...");
        System.out.println("Nuevo tamaño de la playlist: " +
playlist.size()); // Imprime: 3

        // Recorremos la colección con un bucle for-each para ver qué
queda dentro
        System.out.println("Canciones actuales:");
        for (String cancion : playlist) {
            System.out.println("- " + cancion);
        }

        // 7. Método clear() - Vaciamos la colección por completo
        playlist.clear();
        System.out.println("\nVacizando la playlist con clear()...");
        System.out.println("¿Está vacía ahora?: " + playlist.isEmpty());
// Imprime: true
    }
}

```

Ejemplo usando HashSet

```
import java.util.HashSet;
import java.util.Collection;

public class EjemploCollectionSet {
    public static void main(String[] args) {

        // 1. Declaramos el tipo Interfaz (Collection) e instanciamos un
        // HashSet
        Collection<Integer> codigosAcceso = new HashSet<>();

        // Comprobación inicial con isEmpty()
        System.out.println("¿Hay códigos registrados?: " +
!codigosAcceso.isEmpty()); // Imprime: false

        // 2. Método add(E e) - Añadimos códigos (usamos la clase
        // envoltorio Integer)
        codigosAcceso.add(1024);
        codigosAcceso.add(5050);
        codigosAcceso.add(9999);

        // ¡Intento duplicado! Intentamos meter el 1024 otra vez
        // Al ser un HashSet, el método add detecta el duplicado y lo
        // rechaza (devuelve false internamente)
        codigosAcceso.add(1024);

        System.out.println("\n--- Registro de Seguridad (Set) ---");

        // 3. Método size() - Cuenta los elementos únicos
        System.out.println("Total de códigos únicos guardados: " +
codigosAcceso.size()); // Imprime: 3 (ignoró el duplicado)

        // 4. Método contains(Object o) - Verificación rápida de acceso
        int codigoAProbar = 5050;
        if (codigosAcceso.contains(codigoAProbar)) {
            System.out.println("Acceso PERMITIDO para el código: " +
codigoAProbar);
        } else {
            System.out.println("Acceso DENEGADO.");
        }

        // 5. Método remove(Object o) - Damos de baja un código
        codigosAcceso.remove(9999);
        System.out.println("\nEliminando el código 9999...");

        // Mostramos el contenido. Verás que el orden en la consola
```

```

puede ser totalmente aleatorio
    System.out.println("Códigos activos en el sistema:");
    for (int codigo : codigosAcceso) {
        System.out.println("- Cod: " + codigo);
    }

    // 6. Método clear() - Borrado de pánico (eliminar todas las
credenciales)
    codigosAcceso.clear();
    System.out.println("\nSistema reseteado con clear(). Total
elementos: " + codigosAcceso.size()); // Imprime: 0
    }
}

```

ArrayList y sus Métodos Propios

Un ArrayList implementa la interfaz List. Por detrás funciona con un array interno que se redimensiona solo, permitiéndote acceder por posición de manera ultra rápida. Aparte de los métodos de Collection, tiene estos métodos indexados:

- `get(int index)`: Recupera el elemento de esa posición (el lista[i] de Python).
- `set(int index, E element)`: Modifica el elemento de esa posición.
- `remove(int index)`: Borra el elemento pasándole su posición.
- `indexOf(Object o)`: Te dice en qué índice está un elemento (o -1 si no existe).

```

import java.util.ArrayList;

public class EjemploMetodosArrayList {
    public static void main(String[] args) {

        // 1. Creamos el ArrayList de Strings para los pilotos
        ArrayList<String> carrera = new ArrayList<>();

        // Usamos el .add() normal heredado de Collection para llenar la
parrilla
        // Posición [0] -> Verstappen, [1] -> Hamilton, [2] -> Alonso
        carrera.add("Verstappen");
        carrera.add("Hamilton");
        carrera.add("Alonso");

        System.out.println("=== PARRILLA DE SALIDA INICIAL ===");
    }
}

```

```

        System.out.println(carrera); // Imprime: [Verstappen, Hamilton,
Alonso]

        //
=====
        // METODO 1: get(int index) -> Recuperar elemento por su
posición
        //
=====
        // Queremos saber quién va en la primera posición (índice 0)
        String lider = carrera.get(0);
        System.out.println("\nEl piloto en la Pole Position es: " +
lider); // Verstappen

        //
=====
        // METODO 2: indexOf(Object o) -> Buscar el índice de un
elemento
        //
=====
        // Queremos saber en qué posición está "Alonso" para controlar
la carrera
        int posicionAlonso = carrera.indexOf("Alonso");
        System.out.println("Alonso está actualmente en el índice: " +
posicionAlonso); // Imprime: 2

        // ¿Qué pasa si buscamos a un piloto que no está corriendo?
        int posicionSainz = carrera.indexOf("Sainz");
        System.out.println("¿Sainz está en la carrera?: " +
posicionSainz); // Imprime: -1 (no existe)

        //
=====
        // METODO 3: set(int index, E element) -> Modificar un elemento
        //
=====
        // ¡Adelantamiento! Alonso adelanta a Hamilton.
        // El piloto del índice 1 pasa a ser "Alonso" (machacando a
Hamilton)
        System.out.println("\n¡Gran maniobra! Alonso adelanta a
Hamilton.");
        carrera.set(1, "Alonso");

```

```

        System.out.println("Parrilla actualizada: " + carrera); //
Imprime: [Verstappen, Alonso, Alonso]
        // NOTA: Como 'set' sobrescribe, Hamilton desapareció. (Para
meterlo sin borrar usaríamos otra variante de add).

        //
=====
        // METODO 4: remove(int index) -> Borrar un elemento por su
índice
        //
=====
        // ¡Accidente! El líder Verstappen (índice 0) rompe el motor y
abandona la carrera.
        System.out.println("\n¡Atención! Verstappen rompe el motor y
abandona.");

        // Al borrar el índice 0, automáticamente todo el array se
compacta hacia la izquierda.
        // Lo que antes estaba en el índice 1, ahora pasa a ser el
índice 0.
        carrera.remove(0);

        System.out.println("=== CLASIFICACIÓN FINAL DE LA CARRERA ===");
        // Mostramos las posiciones finales usando un bucle clásico
indexado
        for (int i = 0; i < carrera.size(); i++) {
            System.out.println("Puesto " + (i + 1) + ": " +
carrera.get(i));
        }
        // Imprimiré: Puesto 1: Alonso, Puesto 2: Alonso
    }
}

```

Recorrer Colecciones: For-Each, Iterator y ListIterator

El Bucle For-Each

Es el equivalente exacto al `for` elemento in lista: de Python. Sirve para leer elementos de forma rápida, pero está prohibido usarlo si quieres borrar elementos mientras recorres la lista (Java lanzará una excepción `ConcurrentModificationException`).

```
ArrayList<String> nombres = new ArrayList<>();
nombres.add("Ana");
nombres.add("Carlos");

// Por cada 'nombre' de tipo String dentro de 'nombres'...
for (String nombre : nombres) {
    System.out.println("Nombre: " + nombre);
}
```

Iterator (El estándar para borrar de forma segura)

Un Iterator es un objeto "puntero" que va saltando de elemento en elemento de la lista. Tiene tres métodos clave: `hasNext()` (¿hay un elemento delante?), `next()` (dame el elemento y avanza) y `remove()` (borra el último elemento devuelto). Es la única forma segura de borrar elementos en mitad de un bucle.

```
import java.util.ArrayList;
import java.util.Iterator;

public class EjemploIterator {
    public static void main(String[] args) {
        ArrayList<String> lista = new ArrayList<>();
        lista.add("Vigo");
        lista.add("Cangas");
        lista.add("Moaña");

        // Solicitamos el iterador a la lista
        Iterator<String> it = lista.iterator();

        // Bucle típico de examen: mientras haya elementos delante...
        while (it.hasNext()) {
            String pueblo = it.next(); // Avanza y recupera el dato

            // 🔥 BORRADO SEGURO: Si el pueblo es Cangas, lo borramos de
```



```

la lista
        if (pueblo.equals("Cangas")) {
            it.remove(); // Borra a través del iterador, no de la
lista
        }
    }
    System.out.println(lista); // Imprime: [Vigo, Moaña]
}
}

```

ListIterator (Bidireccional y permite AGREGAR)

Es un hermano mayor del Iterator común. Solo funciona con listas (List) y tiene superpoderes: permite recorrer la lista tanto hacia adelante (hasNext(), next()) como hacia atrás (hasPrevious(), previous()), y además te permite modificar o añadir elementos en mitad del paseo usando add(E e) y set(E e)

```

import java.util.ArrayList;
import java.util.ListIterator;

public class EjemploListIterator {
    public static void main(String[] args) {
        ArrayList<String> letras = new ArrayList<>();
        letras.add("A");
        letras.add("C");

        // Solicitamos el ListIterator
        ListIterator<String> lit = letras.listIterator();

        while (lit.hasNext()) {
            String letra = lit.next();

            // 🛠️ AGREGAR ELEMENTOS EN MITAD DEL RECORRIDO
            if (letra.equals("A")) {
                lit.add("B"); // Añade la "B" justo después de la "A"
            }
        }
        System.out.println(letras); // Imprime: [A, B, C]
    }
}

```

Modificando

```
import java.util.ArrayList;
import java.util.ListIterator;

public class GestionTienda {

    // NUESTRO MÉTODO: Recibe la lista, el texto a buscar y el nuevo
    texto
    public static void actualizarProducto(ArrayList<String> lista,
String buscar, String reemplazo) {
        // Pedimos el iterador de la lista que nos han pasado
        ListIterator<String> it = lista.listIterator();

        while (it.hasNext()) {
            String articulo = it.next(); // Avanzamos el puntero y
leemos

            // Si coincide, modificamos de forma segura mediante el
iterador
            if (articulo.equals(buscar)) {
                it.set(reemplazo); // Modifica el último elemento
devuelto por next()
            }
        }
    }

    public static void main(String[] args) {
        ArrayList<String> inventario = new ArrayList<>();
        inventario.add("Teclado Mecánico v1");
        inventario.add("Ratón Óptico");
        inventario.add("Teclado Mecánico v1"); // Duplicado para probar
que los cambia todos

        System.out.println("Antes: " + inventario);

        // Invocamos nuestro método pasándole los datos necesarios
        actualizarProducto(inventario, "Teclado Mecánico v1", "Teclado
Mecánico v2");

        System.out.println("Después: " + inventario);
        // Imprime: [Teclado Mecánico v2, Ratón Óptico, Teclado Mecánico
v2]
    }
}
```

Ejer SOCIO

Clase Persona

```
public class Persona {

    // ATRIBUTOS: Definimos las características de la persona
    public String nombre;
    public int edad;
    public String dni;

    // CONSTRUCTOR: Se ejecuta al hacer 'new Persona(...)'
    public Persona(String nombre, int edad, String dni) {
        this.nombre = nombre;
        this.edad = edad;
        this.dni = dni;
    }

    // GETTERS: Métodos para obtener los datos privados desde fuera
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nuevoNombre) {
        this.nombre = nuevoNombre;
    }

    public int getEdad() {
        return edad;
    }
    public void setEdad(int nuevoEdad) {
        this.edad = nuevoEdad;
    }

    public String getDni() {
        return dni;
    }

    public String toString() {
        return "Nome: " + nombre + " | Idade: " + edad + " | DNI: "
+ dni;
    }

    public int compareTo(Persona outra) {
        // Usamos el compareTo que ya tienen los Strings por
defecto
        int resultadoCom = this.dni.compareTo(outra.dni);
```

```

        // REGLA DE ORO DE COMPARABLE:
        // Si devuelve < 0: 'this' va antes que 'outra'
        // Si devuelve > 0: 'this' va después que 'outra'
        // Si devuelve 0: son iguales

        if (resultadoCom > 0) {
            return 1;
        } else if (resultadoCom < 0) {
            return -1; // Corregido: -1 para indicar que es menor
        }
        return 0; // Son iguales
    }

    public boolean equals(Object obj) {
        Persona outraPessoa = (Persona) obj;
        if(this.dni.equals(outraPessoa.dni)) return true;
        return false;
    }
}

```

Clase Socio

```

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class Socio extends Persona{
    LocalDate dateAlta;

    public Socio(String nombre, int edad, String dni, String
dAlta){
        super(nombre,edad,dni);
        DateTimeFormatter f =
DateTimeFormatter.ofPattern("dd/MM/yyyy");
        dateAlta = LocalDate.parse(dAlta,f);
    }

    public int antiguedade(){

        return (int)dateAlta.until(LocalDate.now(),
ChronoUnit.YEARS);
    }

    public String toString() {
        return "Nome: " + nombre + " | Idade: " + edad + " | DNI: "
+ dni + " | Data: " + dateAlta;
    }
}

```

```
}
```

Clase Gestor

```
import java.util.*;
import java.util.Iterator;

public class GestorSocios {
    public List<Socio> ListaSocios;

    public GestorSocios() {
        ListaSocios = new ArrayList<>();
    }

    public boolean agregarSocio(Socio nuevoSocio){
        return ListaSocios.add(nuevoSocio);
    }

    public boolean bajaSocio(String dniBuscar){
        /*
        for (int i =0; i < this.ListaSocios.size(); i++){
            if(ListaSocios.get(i).dni.equals(dniBuscar)) {
                ListaSocios.remove(i);
                return true;
            }
        }
        */
        /*
        Iterator<Socio> it = ListaSocios.iterator();
        while (it.hasNext()){
            Socio s = it.next();
            if(s.dni.equals(dniBuscar)){
                it.remove();
                return true;
            }
        }
        */
        Socio outro = new Socio(" ",0,"0000000T","01/01/0001");
        ListaSocios.remove(outro);
        return true;
    }

    public void modificarNombre(String nombreBuscar, String
nuevoNombre) {
        ListIterator<Socio> it = this.ListaSocios.listIterator();
        while (it.hasNext()){
            Socio s = it.next();
            if (s.getNombre().equals(nombreBuscar)) {
```

```

        s.setNombre(nuevoNombre);
    }
}

// NUESTRO MeTODO: Muestra toda la lista, pero SOLO el DNI de
cada uno
public void mostrarSoloDnis() {

    System.out.println("=== LISTADO EXCLUSIVO DE DNIS ===");

    // Recorremos la lista de socios uno por uno
    for (Socio s : this.ListaSocios) {

        // LA CLAVE: Imprimimos únicamente el DNI usando su
getter
        System.out.println(s.getDni());

    }
}

// NUESTRO MeTODO: Muestra toda la lista, pero SOLO el DNI de
cada uno
public void mostrarSoloAntiguedad() {

    System.out.println("=== LISTADO EXCLUSIVO DE antigüedad
===");

    // Recorremos la lista de socios uno por uno
    for (Socio s : this.ListaSocios) {
        System.out.println("Años de antigüedad" +
s.antigüedad());
    }
}
}

```

Clase Menu

```
import java.util.Scanner;

public class Menu {

    public static void main(String[] args) {
        GestorSocios gestor = new GestorSocios();
        Scanner teclado = new Scanner(System.in);
        int opcion = 0;

        do {
            System.out.println("\n=== GESTIÓN DE SOCIOS ===");
            System.out.println("1. Dar de alta socio");
            System.out.println("2. Dar de baja socio");
            System.out.println("3. Modificar nombre de un socio");
            System.out.println("4. Mostrar listado de DNI");
            System.out.println("5. Mostrar listado de antigüedad");
            System.out.println("6. Salir");
            System.out.print("Seleccione una opción: ");

            // Lee el número directamente sin try-catch
            opcion = Integer.parseInt(teclado.nextLine());

            switch (opcion) {
                case 1:
                    System.out.println("\n--- ALTA DE SOCIO ---");
                    System.out.print("Introduce el nombre: ");
                    String nombre = teclado.nextLine();
                    System.out.print("Introduce la edad: ");
                    int edad = Integer.parseInt(teclado.nextLine());
                    System.out.print("Introduce el DNI: ");
                    String dni = teclado.nextLine();
                    System.out.print("Introduce la fecha de alta
(dd/MM/yyyy): ");
                    String fecha = teclado.nextLine();

                    Socio nuevo = new Socio(nombre, edad, dni,
fecha);
                    gestor.agregarSocio(nuevo);
                    System.out.println("¡Socio dado de alta con
éxito!");
                    break;

                case 2:
```

```
        System.out.println("\n--- BAJA DE SOCIO ---");
        System.out.print("Introduce el DNI del socio a
dar de baja: ");
        String dniBaja = teclado.nextLine();
        if (gestor.bajaSocio(dniBaja)) {
            System.out.println(";Socio eliminado
correctamente!");
        } else {
            System.out.println("No se encontró ningún
socio con ese DNI.");
        }
        break;

    case 3:
        System.out.println("\n--- MODIFICAR NOMBRE
---");
        System.out.print("Introduce el nombre actual
que deseas buscar: ");
        String nombreBuscar = teclado.nextLine();
        System.out.print("Introduce el nuevo nombre:
");
        String nuevoNombre = teclado.nextLine();

        gestor.modificarNombre(nombreBuscar,
nuevoNombre);
        System.out.println("Proceso de modificación
completado.");
        break;

    case 4:
        System.out.println();
        gestor.mostrarSoloDnis();
        break;

    case 5:
        System.out.println();
        gestor.mostrarSoloAntiguedad();
        break;

    case 6:
        System.out.println("\nSaliendo del programa...
;Hasta pronto!");
        break;

    default:
        System.out.println("Opción no válida. Por
favor, elige un número del 1 al 6.");
        break;
```



```

    }

    } while (opcion != 6);

    teclado.close();
}
}

```

Random

Codificar unha aplicación onde se insertan 100 números enteiros aleatorios entre 1 e 10 (ámbolos dous incluídos), podendo estar repetidos, nunha colección. Despois elimínanse tódolos elementos que valen 5 e 7. Mostrar a colección antes e despois da eliminación

```

import java.util.Random;

import java.util.ArrayList;

import java.util.List;

import java.util.Iterator;

public class NumerosRandom {

    public static void main(String[] args) {

        // 1. Llamamos al metodo para crear la lista y
guardamos el resultado

        List<Integer> miLista = crearRandom();

        // 2. Pasamos esa lista al metodo de eliminar

        eliminar5y7(miLista);

    }
}

```

```

    // Cambiamos 'void' por 'List<Integer>' para que el main
reciba la lista

    public static List<Integer> crearRandom() {

        List<Integer> numeros = new ArrayList<>();

        Random random = new Random();

        for (int i = 0; i < 100; i++) {

            numeros.add(random.nextInt(10) + 1);

        }

        System.out.println("Lista antes de eliminar os 5 e os
7:");

        System.out.println(numeros);

        return numeros; // ;Importante! Devolvemos la lista
creada

    }

    // Añadimos 'static' para que el main pueda usarlo

    public static void eliminar5y7(List<Integer> numeros) {

        // Usamos un bucle hacia atrás para evitar el error de
saltarse índices al borrar

        /*No va

        for (int i =0; i < numeros.size(); i++) {

            if (numeros.get(i) == 5 || numeros.get(i) == 7) {

                numeros.remove(i);

            }

        }

```

```
}

*/

/*Va con chapuza

for (int i =0; i < numeros.size(); i++) {

    if (numeros.get(i) == 5 || numeros.get(i) == 7) {

        numeros.remove(i);

    }

    i--;

}

*/

// Va con chapuza

Iterator<Integer> it = numeros.iterator();

while (it.hasNext()) {

    Integer n = it.next();

    if (n == 5 || n == 7) it.remove();

}

System.out.println("\nLista depois de eliminar os 5 e
os 7:");

System.out.println(numeros);

}

}
```

Base

1. Conexión (Con Try-with-resources)

Este ejemplo muestra cómo conectarse a la base de datos PostgreSQL utilizando el enfoque moderno y seguro: Try-with-resources. De esta forma, nos aseguramos de que la conexión se cierre automáticamente (gracias a AutoCloseable) sin necesidad de un bloque finally engorroso.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionEjemplo {
    public static void main(String[] args) {
        // Datos de la conexión basados en tus apuntes
        String url = "jdbc:postgresql://10.0.8.165:5432/usuarios";
        String usuario = "tu_usuario";
        String contrasena = "tu_contrasena";

        // Al declarar la conexión dentro del try, Java la cerrará automáticamente al terminar
        try (Connection conexion = DriverManager.getConnection(url, usuario, contrasena)) {

            if (conexion != null) {
                System.out.println(" Conexión establecida con éxito a PostgreSQL.");
            }

        } catch (SQLException e) {
            System.err.println("Error al conectar a la base de datos: " + e.getMessage());
        }
    }
}
```

Para interactuar con una base de datos relacional (como PostgreSQL) desde Java, se utiliza la API JDBC (Java Database Connectivity).

Componentes Clave:

- Connection: Gestiona la sesión y la comunicación directa con la base de datos.
- DriverManager.getConnection(url, user, pass): Establece el enlace físico.
- Estructura de la URL de conexión (PostgreSQL):
jdbc:postgresql://[IP_o_Host]:[Puerto]/[Nombre_BD]
Ejemplo: "jdbc:postgresql://10.0.8.165:5432/usuarios"

2. Statement vs PreparedStatement (Ejemplos Reales)

Statement

Se utiliza para consultas SQL fijas y estáticas (como la creación de tablas).

```
Statement sentencia = conexion.createStatement();
sentencia.execute("CREATE TABLE..."); // Devuelve un boolean
sentencia.executeUpdate("INSERT INTO..."); // Devuelve filas afectadas
```

PreparedStatement (¡La opción recomendada!)

Se utiliza para consultas dinámicas que reciben parámetros.

- Ventajas: Previene ataques de Inyección SQL y mejora el rendimiento gracias a la precompilación.
- Sintaxis: Utiliza marcadores de posición (?) cuyo índice empieza en 1.

```
Statement sentencia = conexion.createStatement();
sentencia.execute("CREATE TABLE..."); // Devuelve un boolean
sentencia.executeUpdate("INSERT INTO..."); // Devuelve filas afectadas
```

Aquí verás la diferencia en acción. Usaremos Statement para crear una tabla (estructura fija) y PreparedStatement para insertar datos de forma dinámica y segura contra Inyección SQL.

```
import java.sql.Connection;
```

```

import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class SentenciasEjemplo {
    public static void main(String[] args) {
        String url = "jdbc:postgresql://10.0.8.165:5432/usuarios";
        String user = "postgres";
        String pass = "1234";

        try (Connection conexion = DriverManager.getConnection(url, user, pass)) {

            // --- 1. EJEMPLO CON STATEMENT (Consulta estática) ---
            try (Statement sentencia = conexion.createStatement()) {
                String sqlCrearTabla = "CREATE TABLE IF NOT EXISTS pessoas (" +
                    "nome VARCHAR(50), " +
                    "dni VARCHAR(9) PRIMARY KEY, " +
                    "edade INT)";

                // execute() devuelve boolean (false porque es un comando DDL que no
                // devuelve filas)
                sentencia.execute(sqlCrearTabla);
                System.out.println("Tabla 'pessoas' verificada/creada correctamente.");
            }

            // --- 2. EJEMPLO CON PREPAREDSTATEMENT (Consulta dinámica con
            // parámetros) ---
            String sqlInsertar = "INSERT INTO pessoas(nome, dni, idade) VALUES (?, ?, ?)";

            try (PreparedStatement ps = conexion.prepareStatement(sqlInsertar)) {
                // Asignamos los valores sustituyendo los marcadores '?' (El índice empieza en
                // 1)
                ps.setString(1, "Roque"); // Primer '?'
                ps.setString(2, "9876D"); // Segundo '?'
                ps.setInt(3, 41); // Tercer '?'

                // executeUpdate() devuelve el número de filas afectadas
                int filasAfectadas = ps.executeUpdate();
                System.out.println("Filas insertadas correctamente: " + filasAfectadas);
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

3. Avanzado: Configuración del ResultSet y Navegación

Por defecto, un ResultSet es de tipo TYPE_FORWARD_ONLY (solo avanza) y CONCUR_READ_ONLY (no modificable). Para alterar este comportamiento, se debe configurar el Statement o PreparedStatement pasándole parámetros específicos.

Tipos de ResultSet (tipoResultSet):

- ResultSet.TYPE_FORWARD_ONLY: El cursor solo puede moverse hacia adelante (de arriba a abajo). Es el comportamiento por defecto.
- ResultSet.TYPE_SCROLL_INSENSITIVE: El cursor se puede mover bidireccionalmente (atrás/adelante) y saltar a posiciones absolutas. No refleja los cambios hechos por otros usuarios en la BD mientras está abierto.
- ResultSet.TYPE_SCROLL_SENSITIVE: Permite movimiento bidireccional y sí refleja en tiempo real los cambios que otros usuarios hagan en los datos del buffer.

Tipos de Concurrencia (concurrency):

- ResultSet.CONCUR_READ_ONLY: Solo lectura. No se pueden modificar los datos desde el objeto ResultSet.
- ResultSet.CONCUR_UPDATABLE: Permite modificar, eliminar o insertar filas directamente en el objeto ResultSet y sincronizar esos cambios en la base de datos física.

Métodos de Navegación del Cursor:

- next(): Avanza a la siguiente fila. Devuelve false si no hay más filas.

```
rs.beforeFirst(); // Posición inicial

if (rs.next()) {
    System.out.println(rs.getString("nome")); // Imprime "Roque" (Fila 1)
}
if (rs.next()) {
    System.out.println(rs.getString("nome")); // Imprime "Ana" (Fila 2)
}
if (rs.previous()) {
    System.out.println(rs.getString("nome")); // Imprime "Roque" de nuevo (Fila 1)
}
```

- previous(): Retrocede a la fila anterior (requiere tipo Scrollable).

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class EjemploPrevious {
    public static void main(String[] args) {
        String url = "jdbc:postgresql://10.0.8.165:5432/usuarios";
        String user = "postgres";
        String pass = "1234";
        String sql = "SELECT nome FROM pessoas";

        try (Connection conexion = DriverManager.getConnection(url, user, pass);
            // REQUISITO DE EXAMEN CRÍTICO:
            // Si dejas el PreparedStatement por defecto, rs.previous() lanzará una
            SQLException.
            // Necesitamos obligatoriamente 'TYPE_SCROLL_INSENSITIVE' o
            'TYPE_SCROLL_SENSITIVE'.
            PreparedStatement ps = conexion.prepareStatement(
                sql,
                ResultSet.TYPE_SCROLL_INSENSITIVE,
                ResultSet.CONCUR_READ_ONLY);
            ResultSet rs = ps.executeQuery()) {

            System.out.println("--- Avanzamos primero para posicionar el cursor ---");

            if (rs.next()) { // Avanza a Fila 1
                System.out.println("Fila " + rs.getRow() + ": " + rs.getString("nome")); // Roque
            }
            if (rs.next()) { // Avanza a Fila 2
                System.out.println("Fila " + rs.getRow() + ": " + rs.getString("nome")); // Ana
            }
            if (rs.next()) { // Avanza a Fila 3
                System.out.println("Fila " + rs.getRow() + ": " + rs.getString("nome")); // Carlos
            }

            System.out.println("\n--- Probando rs.previous() hacia atrás ---");

            // 1. Primer retroceso (De Fila 3 a Fila 2)
            // rs.previous() devuelve true porque se movió con éxito a una fila válida.
            boolean movidoA2 = rs.previous();
            System.out.println("¿Se pudo retroceder?: " + movidoA2);
            System.out.println("Fila actual: " + rs.getRow() + " | Nombre: " +
rs.getString("nome")); // Ana

```



```

// 2. Segundo retroceso (De Fila 2 a Fila 1)
boolean movidoA1 = rs.previous();
System.out.println("¿Se pudo retroceder?: " + movidoA1);
System.out.println("Fila actual: " + rs.getRow() + " | Nombre: " +
rs.getString("nome")); // Roque

// 3. Tercer retroceso (De Fila 1 a 'beforeFirst')
// Aquí salimos de los límites de los datos. El método devuelve false.
System.out.println("\n--- Caso Límite: Intentar retroceder más allá de la primera fila
---");
boolean movidoFuera = rs.previous();

System.out.println("¿Se pudo retroceder?: " + movidoFuera); // Imprime: false
System.out.println("¿Está el cursor en beforeFirst?: " + rs.isBeforeFirst()); //
Imprime: true

// OJO EN EL EXAMEN:
// Si intentas hacer rs.getString("nome") aquí, el programa lanzará una
SQLException
// porque el cursor está en una zona muerta antes de la primera fila.

} catch (SQLException e) {
    System.err.println("Error de base de datos: " + e.getMessage());
    e.printStackTrace();
}
}
}

```

- `absolute(int fila)`: Mueve el cursor a un número de fila exacto (ej. `absolute(1)` va a la primera fila).

```

// Ir a la primera fila
if (rs.absolute(1)) {
    System.out.println(rs.getString("nome")); // Imprime "Roque"
}

// Ir a la tercera fila
if (rs.absolute(3)) {
    System.out.println(rs.getString("nome")); // Imprime "Carlos"
}

// TRUCO DE EXAMEN: Números negativos
// absolute(-1) se posiciona siempre en la ÚLTIMA fila del ResultSet
if (rs.absolute(-1)) {
    System.out.println(rs.getString("nome")); // Imprime "Carlos"
}

```

```
// absolute(-2) se posiciona en la penúltima fila
if (rs.absolute(-2)) {
    System.out.println(rs.getString("nome")); // Imprime "Ana"
}
```

- relative(int filas): Mueve el cursor el número de filas indicado relativo a donde esté (positivo hacia adelante, negativo hacia atrás).

```
rs.absolute(1); // Estamos en "Roque" (Fila 1)

// Nos movemos 2 filas hacia adelante (1 + 2 = Fila 3)
if (rs.relative(2)) {
    System.out.println(rs.getString("nome")); // Imprime "Carlos"
}

// Nos movemos 1 fila hacia atrás (3 - 1 = Fila 2)
if (rs.relative(-1)) {
    System.out.println(rs.getString("nome")); // Imprime "Ana"
}
```

- beforeFirst() / afterLast(): Coloca el cursor antes del primer registro o después del último.

```
// El cursor por defecto empieza en beforeFirst().
// Si queremos asegurarnos de volver arriba del todo antes de un bucle:
rs.beforeFirst();

// Si queremos empezar a leer la base de datos al revés,
// primero saltamos al final de todo (justo después de Carlos)
rs.afterLast();
while (rs.previous()) {
    System.out.println(rs.getString("nome")); // Imprimirá: Carlos -> Ana -> Roque
}
```

- isFirst() / isLast() / isBeforeFirst() / isAfterLast(): Métodos booleanos de comprobación de posición.

```
rs.absolute(1); // Nos paramos en "Roque"

if (rs.isFirst()) {
    System.out.println("Efectivamente, estamos en la primera fila.");
}

rs.afterLast(); // Vamos al final del todo (fuera de los registros)

if (rs.isAfterLast()) {
    System.out.println("El cursor pasó el último registro, no hay más datos aquí.");
}
```

```
}
```

```
// Si intentas leer datos aquí:
```

```
// rs.getString("nome"); -> Lanza SQLException porque no es una fila válida.
```

Ejemplo avanzado

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ResultSetAvanzadoEjemplo {
    public static void main(String[] args) {
        String url = "jdbc:postgresql://10.0.8.165:5432/usuarios";

        String sql = "SELECT nome, dni, idade FROM pessoas";

        try (Connection conexion = DriverManager.getConnection(url, "postgres", "1234");
            // CONFIGURACIÓN CLAVE: Pasamos el tipo de Scroll y la Concurrencia al
            preparar la sentencia
            PreparedStatement ps = conexion.prepareStatement(
                sql,
                ResultSet.TYPE_SCROLL_INSENSITIVE,
                ResultSet.CONCUR_READ_ONLY);

            ResultSet rs = ps.executeQuery()) {

            System.out.println("--- DEMOSTRACIÓN DE NAVEGACIÓN ---");

            // 1. Ir directamente a la última fila
            if (rs.last()) {
                System.out.println("Última fila -> Nome: " + rs.getString("nome") + " | Posición: "
                    + rs.getRow());
            }

            // 2. Retroceder a la fila anterior
            if (rs.previous()) {
```

```

        System.out.println("Fila anterior -> Nome: " + rs.getString("nome"));
    }

    // 3. Saltar a una fila absoluta (ej: la primera fila del ResultSet)
    if (rs.absolute(1)) {
        System.out.println("Primera fila absoluta -> Nome: " + rs.getString("nome"));
    }

    // 4. Movimiento relativo (avanzar 2 posiciones desde donde estamos ahora)
    if (rs.relative(2)) {
        System.out.println("Moviendo 2 posiciones adelante -> Nome: " +
rs.getString("nome"));
    }

    // 5. Comprobaciones de posición
    rs.afterLast(); // Ponemos el cursor justo DESPUÉS de la última fila
    if (rs.isAfterLast()) {
        System.out.println("El cursor se encuentra efectivamente después del último
registro.");
    }

    // 6. Volver al principio para recorrerlo de forma normal
    rs.beforeFirst(); // Posiciona el cursor ANTES de la primera fila
    System.out.println("\n--- Recorrido secuencial relanzado ---");
    while (rs.next()) {
        System.out.println("- " + rs.getString("nome") + " (" + rs.getInt("edade") + "
años)");
    }

    } catch (SQLException e) {
        System.err.println("Error de base de datos: " + e.getMessage());
    }
}
}

```

Ejemplo avanzado 2

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class EjemploPrevious {
    public static void main(String[] args) {
        String url = "jdbc:postgresql://10.0.8.165:5432/usuarios";
        String user = "postgres";
        String pass = "1234";
        String sql = "SELECT nome FROM pessoas";

        try (Connection conexion = DriverManager.getConnection(url, user, pass);
            // REQUISITO DE EXAMEN CRÍTICO:
            // Si dejas el PreparedStatement por defecto, rs.previous() lanzará una
            // SQLException.
            // Necesitamos obligatoriamente 'TYPE_SCROLL_INSENSITIVE' o
            'TYPE_SCROLL_SENSITIVE'.
            PreparedStatement ps = conexion.prepareStatement(
                sql,
                ResultSet.TYPE_SCROLL_INSENSITIVE,
                ResultSet.CONCUR_READ_ONLY);
            ResultSet rs = ps.executeQuery()) {

            System.out.println("--- Avanzamos primero para posicionar el cursor ---");

            if (rs.next()) { // Avanza a Fila 1
                System.out.println("Fila " + rs.getRow() + ": " + rs.getString("nome")); // Roque
            }
            if (rs.next()) { // Avanza a Fila 2
                System.out.println("Fila " + rs.getRow() + ": " + rs.getString("nome")); // Ana
            }
            if (rs.next()) { // Avanza a Fila 3
                System.out.println("Fila " + rs.getRow() + ": " + rs.getString("nome")); // Carlos
            }
        }
    }
}
```

```

System.out.println("\n--- Probando rs.previous() hacia atrás ---");

// 1. Primer retroceso (De Fila 3 a Fila 2)
// rs.previous() devuelve true porque se movió con éxito a una fila válida.
boolean movidoA2 = rs.previous();
System.out.println("¿Se pudo retroceder?: " + movidoA2);
System.out.println("Fila actual: " + rs.getRow() + " | Nombre: " +
rs.getString("nome")); // Ana

// 2. Segundo retroceso (De Fila 2 a Fila 1)
boolean movidoA1 = rs.previous();
System.out.println("¿Se pudo retroceder?: " + movidoA1);
System.out.println("Fila actual: " + rs.getRow() + " | Nombre: " +
rs.getString("nome")); // Roque

// 3. Tercer retroceso (De Fila 1 a 'beforeFirst')
// Aquí salimos de los límites de los datos. El método devuelve false.
System.out.println("\n--- Caso Límite: Intentar retroceder más allá de la primera fila
---");
boolean movidoFuera = rs.previous();

System.out.println("¿Se pudo retroceder?: " + movidoFuera); // Imprime: false
System.out.println("¿Está el cursor en beforeFirst?: " + rs.isBeforeFirst()); //
Imprime: true

// OJO EN EL EXAMEN:
// Si intentas hacer rs.getString("nome") aquí, el programa lanzará una
SQLException
// porque el cursor está en una zona muerta antes de la primera fila.

} catch (SQLException e) {
    System.err.println("Error de base de datos: " + e.getMessage());
    e.printStackTrace();
}
}
}

```

Ejemplo avanzado 3

```
// Archivo: PruebaJDBC.java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class PruebaJDBC {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mi_instituto";
        String usuario = "root";
        String password = "password_secreto"; // ¡Pon el tuyo! [cite: 117]

        String sql = "SELECT id, nombre, nota FROM Alumnos WHERE nota >= ?";

        // Usamos try-with-resources para que Java cierre todo automáticamente al terminar
        try (Connection con = DriverManager.getConnection(url, usuario, password);
            PreparedStatement ps = con.prepareStatement(sql)) {

            // 1. Inyectamos el parámetro de forma segura (Evita SQL Injection)
            ps.setDouble(1, 5.0); // El primer '?' se cambia por un 5.0 (Aprobados)

            // 2. Ejecutamos y guardamos el camión de datos en el ResultSet
            ResultSet rs = ps.executeQuery();

            System.out.println("--- LISTA DE APROBADOS ---");

            // 3. Recorremos el ResultSet con nuestro bucle de confianza [cite: 58]
            while (rs.next()) {
                // Sacamos los datos de la fila actual por el nombre de su columna
                int id = rs.getInt("id");
                String nombre = rs.getString("nombre"); [cite: 123]
                double nota = rs.getDouble("nota"); [cite: 124]

                // Imprimimos formateado bien limpio en consola
                System.out.println("ID: " + id + " | Alumno: " + nombre + " | Nota: " + nota);
            }
        }
    }
}
```

```
    } catch (Exception e) {  
        System.out.println("¡Error de conexión o SQL incorrecto!");  
        e.printStackTrace();  
    }  
}  
}
```

Ejemplo TRABAJADO EN CLASE

Clase Persona

```
// La interfaz Comparable permite que objetos de tipo Persona se  
puedan ordenar  
public class Persona implements Comparable<Persona>{  
  
    // ATRIBUTOS: Definimos las características de la persona  
    private String nombre;  
    private int edad;  
    private String dni;  
  
    // CONSTRUCTOR: Se ejecuta al hacer 'new Persona(...)'  
    public Persona(String nombre, int edad, String dni) {  
        this.nombre = nombre;  
        this.edad = edad;  
        this.dni = dni;  
        // Cada vez que nace una persona, sumamos 1 al total  
    }  
  
    // GETTERS: Métodos para obtener los datos privados desde fuera  
    public String getNombre() {  
        return nombre;  
    }  
  
    public int getEdad() {  
        return edad;  
    }  
}
```



```

    public String getDni() {
        return dni;
    }

    @Override
    public String toString() {
        return "Nome: " + nombre + " | Idade: " + edad + " | DNI: "
+ dni;
    }
    @Override
    public int compareTo(Pessoa outra) {
        // Usamos el compareTo que ya tienen los Strings por
defecto
        int resultadoCom = this.dni.compareTo(outra.dni);

        // REGLA DE ORO DE COMPARABLE:
        // Si devuelve < 0: 'this' va antes que 'outra'
        // Si devuelve > 0: 'this' va después que 'outra'
        // Si devuelve 0: son iguales

        if (resultadoCom > 0) {
            return 1;
        } else if (resultadoCom < 0) {
            return -1; // Corregido: -1 para indicar que es menor
        }
        return 0; // Son iguales
    }

    // igualador de DNI
    @Override
    public boolean equals(Object obj) {
        Pessoa outraPessoa = (Pessoa) obj;
        if(this.dni.equals( outraPessoa.dni)) return true;
        return false;
    }
}

```

UsuarioDAO

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class UsuarioDAO {

    // Configuración de la base de datos como constantes
    private static final String URL =
"jdbc:postgresql://10.0.8.165:5432/usuarios";
    private static final String USER = "vboxuser";
    private static final String PASS = "vboxuser";

    private static Connection conectarBD() {
        Connection conexion = null;
        try {
            conexion = DriverManager.getConnection(URL, USER, PASS);
        } catch (SQLException e) {
            System.out.println("Error al conectar a la BD: " +
e.getMessage());
        }
        return conexion;
    }

    // 1. INSERTAR
    public static boolean crearUsuarioBD(Pessoa p) {
        // 1. VALIDACIÓN: Buscamos si el DNI ya existe en la base
de datos
        Pessoa personaExistente = buscarPorDniBD(p.getDni());

        if (personaExistente != null) {
```

```

        System.out.println("Error: Ya existe una persona
registrada con el DNI " + p.getDni() + " (" +
personaExistente.getNombre() + ").");
        return false; // Cancelamos la inserción
    }

    // 2. INSERCIÓN: Si no existe, procedemos a guardarlo
    String sql = "INSERT INTO persoas(nome, dni, idade) VALUES
(?, ?, ?)";

    try (Connection con = conectarBD()) {
        if (con == null) return false;

        try (PreparedStatement sentenciaP =
con.prepareStatement(sql)) {
            sentenciaP.setString(1, p.getNombre());
            sentenciaP.setString(2, p.getDni());
            sentenciaP.setInt(3, p.getEdad());

            int filasAfectadas = sentenciaP.executeUpdate();
            return filasAfectadas > 0;
        }
    } catch (SQLException e) {
        System.out.println("Error al insertar la pessoa: " +
e.getMessage());
        return false;
    }
}

// 2. LISTAR TDO (Devuelve una lista de objetos Pessoa)
public static List<Pessoa> listarPersoasBD() {
    List<Pessoa> lista = new ArrayList<>();
    String sql = "SELECT nome, dni, idade FROM persoas";

    try (Connection con = conectarBD()) {
        if (con == null) return lista;

        try (PreparedStatement sentenciaP =
con.prepareStatement(sql);
            ResultSet resultados = sentenciaP.executeQuery())
        {
            while (resultados.next()) {
                // Creamos un objeto Pessoa por cada fila de la
                Pessoa p = new Pessoa(
                    resultados.getString("nome"),
                    resultados.getInt("idade"),

```

```

        resultados.getString("dni")
    );
    lista.add(p); // Lo añadimos a la lista
}
}
} catch (SQLException e) {
    System.out.println("Error al listar persoas: " +
e.getMessage());
}
return lista;
}

// 3. BUSCAR POR DNI (Devuelve una Pessoa o null si no existe)
public static Pessoa buscarPorDniBD(String dniBuscar) {
    String sql = "SELECT nome, dni, idade FROM persoas WHERE
dni = ?";

    try (Connection con = conectarBD()) {
        if (con == null) return null;

        try (PreparedStatement sentenciaP =
con.prepareStatement(sql)) {
            sentenciaP.setString(1, dniBuscar);

            try (ResultSet resultados =
sentenciaP.executeQuery()) {
                if (resultados.next()) {
                    // Si lo encuentra, construye el objeto
Pessoa y lo devuelve
                    return new Pessoa(
                        resultados.getString("nome"),
                        resultados.getInt("idade"),
                        resultados.getString("dni")
                    );
                }
            }
        } catch (SQLException e) {
            System.out.println("Error al buscar por DNI: " +
e.getMessage());
        }
        return null; // Si no lo encuentra o hay error, devuelve
null
    }

    public static void mostrarPessoasAlRevesSQL() {
        // Ordenamos por nombre de la Z a la A (Descendente)
        String sql = "SELECT nome, dni, idade FROM persoas ORDER BY
nome DESC";
    }
}

```

```

        try (Connection con = conectarBD()) {
            if (con == null) return;

            try (PreparedStatement sentenciaP =
con.prepareStatement(sql);
                ResultSet resultados = sentenciaP.executeQuery())
            {
                System.out.println("--- Registros de la BD
(Ordenados por SQL DESC) ---");
                // Leemos hacia adelante normalmente, pero el
contenido ya viene invertido
                while (resultados.next()) {
                    System.out.println("Nome: " +
resultados.getString("nome")
                        + " | DNI: " +
resultados.getString("dni")
                        + " | Edade: " +
resultados.getInt("edade"));
                }
            }
        } catch (SQLException e) {
            System.out.println("Error al consultar: " +
e.getMessage());
        }
    }

    // 4. ELIMINAR POR DNI (Devuelve true si eliminó el registro,
false si no existía o hubo error)
    public static boolean eliminarPorDniBD(String dniEliminar) {
        String sql = "DELETE FROM persoas WHERE dni = ?";

        try (Connection con = conectarBD()) {
            if (con == null) return false;

            try (PreparedStatement sentenciaP =
con.prepareStatement(sql)) {
                // Asignamos el DNI al parámetro '?'
                sentenciaP.setString(1, dniEliminar);

                // executeUpdate devuelve el número de filas
afectadas
                int filasAfectadas = sentenciaP.executeUpdate();

                if (filasAfectadas > 0) {
                    System.out.println("Registro con DNI " +
dniEliminar + " eliminado correctamente.");
                    return true;
                }
            }
        }
    }

```

```

        } else {
            System.out.println("No se encontró a nadie con
el DNI " + dniEliminar + ". Nada que eliminar.");
            return false;
        }
    }
}
} catch (SQLException e) {
    System.out.println("Error al intentar eliminar: " +
e.getMessage());
    return false;
}
}

// 6. ACTUALIZAR POR DNI usando CONCUR_UPDATABLE (Cambia el
nombre y la edad de un DNI existente)
public static boolean actualizarPorDniBD(String dniBuscar,
String nuevoNombre, int nuevaEdad) {
    // Es MUY importante seleccionar las columnas que queremos
modificar
    String sql = "SELECT nome, idade FROM pessoas WHERE dni =
?";

    try (Connection con = conectarBD()) {
        if (con == null) return false;

        // Configuramos el PreparedStatement para que permita
actualizar datos (CONCUR_UPDATABLE)
        try (PreparedStatement sentenciaP =
con.prepareStatement(
            sql,
            ResultSet.TYPE_SCROLL_INSENSITIVE,
            ResultSet.CONCUR_UPDATABLE)) {

            sentenciaP.setString(1, dniBuscar);

            try (ResultSet resultados =
sentenciaP.executeQuery()) {
                // .next() nos posiciona en la fila del usuario
encontrado

                if (resultados.next()) {

                    // 1. Modificamos los valores localmente en
el ResultSet
                    resultados.updateString("nome",
nuevoNombre);

                    resultados.updateInt("idade", nuevaEdad);

                    // 2. Sincronizamos y guardamos los cambios
directamente en la Base de Datos

```

```

        resultados.updateRow();

        System.out.println(" Registro con DNI " +
dniBuscar + " actualizado correctamente a: " + nuevoNombre + " ("
+ nuevaEdad + " años).");
        return true;
    } else {
        System.out.println(" No se encontró a nadie
con el DNI " + dniBuscar + ". No se pudo actualizar.");
        return false;
    }
}

}
} catch (SQLException e) {
    System.out.println("Error al intentar actualizar con
updateRow: " + e.getMessage());
    return false;
}
}

// 5. PRUEBA DE FUNCIONAMIENTO
public static void main(String[] args) {
    // --- Prueba 1: Insertar ---
    String dniPrueba = "9876D";
    Persoa nuevaPersona = new Persoa("Rogue", 41, dniPrueba);
    if (crearUsuarioBD(nuevaPersona)) {
        System.out.println("Guardado correctamente: " +
nuevaPersona.getNombre());
    }

    System.out.println("\n--- Prueba 2: Listar todo ---");
    List<Persoa> todas = listarPersoasBD();
    for (Persoa p : todas) {
        System.out.println("- " + p.getNombre() + " | DNI: " +
p.getDni() + " | Edad: " + p.getEdad());
    }

    System.out.println("\n--- Prueba 3: Buscar por DNI ---");
    String dniABuscar = "9876D";
    Persoa encontrada = buscarPorDniBD(dniABuscar);
    if (encontrada != null) {
        System.out.println("¡Encontrado! Nome: " +
encontrada.getNombre());
    } else {
        System.out.println("No existe ninguna persona con el
DNI: " + dniABuscar);
    }
}

```

```

        System.out.println("\n--- PRUEBA 4: Mostrar al revés usando
SQL (ORDER BY DESC) ---");
        // Muestra los registros ordenados alfabéticamente de la Z
a la A
        mostrarPersoasAlRevesSQL();

        System.out.println("\n--- PRUEBA 5:PEliminando a Roque por
su DNI (" + dniPrueba + ")...");
        boolean eliminado = eliminarPorDniBD(dniPrueba);

        if (eliminado) {
            System.out.println("Éxito: El método confirmó la
eliminación.");
        } else {
            System.out.println("Error: No se pudo eliminar (¿Ya
estaba borrado?).");
        }

        System.out.println("\n--- Prueba 6: Listar todo ---");
        List<Persoa> todas2 = listarPersoasBD();
        for (Persoa p : todas2) {
            System.out.println("- " + p.getNombre() + " | DNI: " +
p.getDni() + " | Edad: " + p.getEdad());
        }
    }
}

```


Conexión

```
import java.sql.Connection;           // La autopista: maneja la
conexión física.
import java.sql.DriverManager;         // El peaje: autentica
usuario/password y te da paso.
import java.sql.SQLException;          // El detector de radares:
captura cualquier fallo de la BD.
import java.sql.Statement;             // El mensajero clásico: para
consultas fijas (estáticas).
import java.sql.PreparedStatement;     // El mensajero blindado: para
consultas con variables (?).
import java.sql.ResultSet;             // El camión repartidor: la
tabla virtual con los datos devueltos.

public class ConexionJava {
    public static void main(String[] args){
        // 1. DECLARACIÓN DE HERRAMIENTAS (Se declaran fuera del
try para tenerlas localizadas)
        Connection conexion;
        Statement sentencia;
```

```

        PreparedStatement sentenciaP;
        ResultSet resultados;

        String sql;
        // La URL de conexión: protocolo + IP de tu Máquina Virtual
+ Puerto Postgres + Nombre de la BD
        String url = "jdbc:postgresql://10.0.8.165:5432/usuarios";

        try {
            // 2. ENTRAR A LA BASE DE DATOS
            // Pasamos la URL, el usuario del sistema ("vboxuser")
y su contraseña ("vboxuser")
            conexion = DriverManager.getConnection(url, "vboxuser",
"vboxuser");
            System.out.println("conectado"); // Si llega aquí, la
tubería está abierta

            // 3. ENFOQUE ESTÁTICO (STATEMENT) - Para crear
estructuras
            String crearTablaSql = "CREATE TABLE pessoas(nome
VARCHAR(50),"
                + "dni VARCHAR (9), idade INTEGER);";

            sentencia = conexion.createStatement(); // Creamos el
mensajero básico

            // ¡OJO! Estas líneas están comentadas porque si las
ejecutas dos veces seguidas,
            // la base de datos petará diciendo "La tabla ya
existe" o duplicarás los datos de Pepe y Ana.
            // sentencia.execute(crearTablaSql);
            // sentencia.executeUpdate("INSERT INTO
pessoas(nome,dni,idade) VALUES ('Pepe','1234J','23')");
            // sentencia.executeUpdate("INSERT INTO
pessoas(nome,dni,idade) VALUES ('Ana','5678L','34')");

            // 4. ENFOQUE DINÁMICO (PREPAREDSTATEMENT) - Para
insertar con seguridad
            // Usamos "?" como comodines para evitar ataques de
Inyección SQL.
            sql = "INSERT INTO pessoas(nome,dni,idade) VALUES
(?,?,?)";
            sentenciaP = conexion.prepareStatement(sql); //
Preparamos la plantilla de la consulta

            // ¡PELIGRO EXAMEN!: Los índices en JDBC empiezan en 1,
NO en 0.

```

```

        sentenciaP.setString(1, "Roque"); // El primer "?" se
cambia por "Roque" (Texto)
        sentenciaP.setString(2, "9876D"); // El segundo "?" se
cambia por el DNI (Texto)
        sentenciaP.setInt(3, 41);          // El tercer "?" se
cambia por la edad (Entero, sin comillas)

        // sentenciaP.executeUpdate(); // <-- Descomentado
metería a Roque en la base de datos.
        // Recordatorio: .executeUpdate() devuelve un 'int' con
las filas afectadas (1 en este caso).

        // 5. EXTRACCIÓN Y LECTURA DE DATOS (SELECT)
        sql = "SELECT nome, dni, edade FROM persoas";
        sentenciaP = conexion.prepareStatement(sql); //
Reutilizamos el mensajero para la nueva consulta

        // Para los SELECT se usa estrictamente
.executeQuery(). Devuelve el camión con los datos (ResultSet).
        resultados = sentenciaP.executeQuery();

        // EXPLICACIÓN DEL PUNTERO: Al recibir 'resultados', el
puntero está en la posición -1 (en el aire).
        // .next() mueve el puntero a la primera fila real.
Devuelve true si hay datos o false si está vacía.

        while (resultados.next()){
            // Sacamos los datos de la fila actual usando el
nombre EXACTO de la columna en Postgres
            String n = resultados.getString("nome");
            String d = resultados.getString("dni");
            int e = resultados.getInt("edade");

            // Lo mostramos en la consola de Java bien
formateado
            System.out.println("Nome: "+ n +", "+ d +", " + e);
        } // El bucle se repite hasta que .next() da false
(cuando se acaban las filas de la tabla)

        // 6. LA HORA DE LA LIMPIEZA (CIERRE DE GRIFOS)
        // Súper estricto: Se cierra en ORDEN INVERSO a como se
abrieron las cosas.
        resultados.close(); // 1º Cerramos el camión de datos
        sentenciaP.close(); // 2º Cerramos el mensajero
preparado
        sentencia.close(); // 3º Cerramos el mensajero básico

```

```

        conexion.close();    // 4º Cortamos el cable de
conexión general por seguridad

    } catch (SQLException e) {
        // Si la IP está mal, la contraseña falla o escribes
"personitas" en vez de "persoas",
        // el código salta directamente aquí y te lanza el
error por consola.
        throw new RuntimeException(e);
    }
}
// Aquí saltarías al Objeto DAO en una aplicación real para no
enguarrar el Main.
}

```

ACtualizar

```

public static void main(String[] args) {
    String dniTest = "1111A";

    // 1. Insertamos un registro de prueba (Pepe con 20 años)
    crearUsuarioBD(new Persoa("Pepe Original", 20, dniTest));

    System.out.println("\n--- PRUEBA ACTUALIZAR (updateRow) ---");
    // 2. Modificamos el nombre y la edad de la persona que tiene
el DNI "1111A"
    boolean actualizado = actualizarPorDniBD(dniTest, "Pepe
Modificado", 99);

    // 3. Comprobamos si el cambio se guardó buscándolo de nuevo
    if (actualizado) {
        Persoa p = buscarPorDniBD(dniTest);
        if (p != null) {
            System.out.println("Resultado en BD -> Nombre: " +
p.getNombre() + " | Edad: " + p.getEdad());
        }
    }
}

```

```
exemplosArray.java  Conexion.java  UsuarioDAO.java x  Main.java  Xogo.java  Pessoa.java  Pessoa2.java
5      public class UsuarioDAO { 3 usages
62      public static void modificarUsuario (String dni, String novoNome, int novaEdade) {
81      }
82      }
83
84      public static void modificarUsuario2 (String dni, String novoNome, int novaEdade) { no usages
85
86          if (dni != null || dni.length() != 0) {
87              Connection conexio = conectaBD(ip: "10.0.0.150", porto: 5432, nomeBD: "usuarios", usuario: "postgres",
88              String sql = "UPDATE usuarios SET nome = ?, edade = ? WHERE dni = ?";
89
90              try {
91                  PreparedStatement sentencia = conexio.prepareStatement(sql);
92                  sentencia.setString(parameterIndex: 1, novoNome);
93                  sentencia.setInt(parameterIndex: 2, novaEdade);
94                  sentencia.setString(parameterIndex: 3, dni);
95                  sentencia.executeUpdate();
96                  conexio.close();
97              } catch (SQLException erro) {
98                  System.out.println("Erro o modificar usuario: " + erro);
99              }
100          }
101      }
102
103
104
105
```

actualizar:

```
// 6. ACTUALIZAR POR DNI usando CONCUR_UPDATABLE (Cambia el nombre
y la edad de un DNI existente)
public static boolean actualizarPorDniBD(String dniBuscar, String
nuevoNombre, int nuevaEdad) {
    // Es MUY importante seleccionar las columnas que queremos
modificar
    String sql = "SELECT nome, edade FROM pessoas WHERE dni = ?";

    try (Connection con = conectarBD()) {
        if (con == null) return false;

        // Configuramos el PreparedStatement para que permita
actualizar datos (CONCUR_UPDATABLE)
        try (PreparedStatement sentenciaP = con.prepareStatement(
            sql,
            ResultSet.TYPE_SCROLL_INSENSITIVE,
            ResultSet.CONCUR_UPDATABLE)) {

            sentenciaP.setString(1, dniBuscar);

            try (ResultSet resultados = sentenciaP.executeQuery())
{
```

```

        // .next() nos posiciona en la fila del usuario
        encontrado

        if (resultados.next()) {

            // 1. Modificamos los valores localmente en el
            ResultSet

            resultados.updateString("nome", nuevoNombre);
            resultados.updateInt("edade", nuevaEdad);

            // 2. Sincronizamos y guardamos los cambios
            directamente en la Base de Datos
            resultados.updateRow();

            System.out.println(" Registro con DNI " +
dniBuscar + " actualizado correctamente a: " + nuevoNombre + " ("
+ nuevaEdad + " años).");
            return true;
        } else {
            System.out.println(" No se encontró a nadie con
el DNI " + dniBuscar + ". No se pudo actualizar.");
            return false;
        }
    }
}

} catch (SQLException e) {
    System.out.println("Error al intentar actualizar con
updateRow: " + e.getMessage());
    return false;
}
}

```

Eliminar

```

// 4. ELIMINAR POR DNI (Devuelve true si eliminó el registro,
false si no existía o hubo error)
public static boolean eliminarPorDniBD(String dniEliminar) {
    String sql = "DELETE FROM pessoas WHERE dni = ?";

    try (Connection con = conectarBD()) {
        if (con == null) return false;

        try (PreparedStatement sentenciaP =
con.prepareStatement(sql)) {
            // Asignamos el DNI al parámetro '?'
            sentenciaP.setString(1, dniEliminar);

            // executeUpdate devuelve el número de filas afectadas
            int filasAfectadas = sentenciaP.executeUpdate();

```

```

        if (filasAfectadas > 0) {
            System.out.println("Registro con DNI " +
dniEliminar + " eliminado correctamente.");
            return true;
        } else {
            System.out.println("No se encontró a nadie con el
DNI " + dniEliminar + ". Nada que eliminar.");
            return false;
        }
    }
} catch (SQLException e) {
    System.out.println("Error al intentar eliminar: " +
e.getMessage());
    return false;
}
}

```

Agregar:

```

// 2. INSERCIÓN: Si no existe, procedemos a guardarlo
String sql = "INSERT INTO persoas(nome, dni, edade) VALUES (?, ?,
?)";

try (Connection con = conectarBD()) {
    if (con == null) return false;

    try (PreparedStatement sentenciaP = con.prepareStatement(sql))
    {
        sentenciaP.setString(1, p.getNombre());
        sentenciaP.setString(2, p.getDni());
        sentenciaP.setInt(3, p.getEdad());

        int filasAfectadas = sentenciaP.executeUpdate();
        return filasAfectadas > 0;
    }
} catch (SQLException e) {
    System.out.println("Error al insertar la persona: " +
e.getMessage());
    return false;
}
}

```

